

Ex No 1

exploring git commands through collaborative coding

Git is a distributed version control system that tracks changes in computer files and coordinates work among multiple people. Created by Linus Torvalds in 2005, it allows developers to safely manage project history, collaborate without overwriting each other's work, and easily revert to previous versions if mistakes happen

Key Concepts

- - **Repository (Repo):** The database where Git stores your project files, history, and branches.
 - **Commit:** A saved "snapshot" of your files at a specific point in time. It acts like a permanent save point in a video game.
 - **Branching & Merging:** A feature that lets you isolate work so you can test new ideas on a "branch" without altering the main codebase. Once finished, you "merge" those changes back into the main project.
 - **Distributed:** Every developer downloads a complete, fully functional local copy of the project and its entire history.

Why Developers Use It

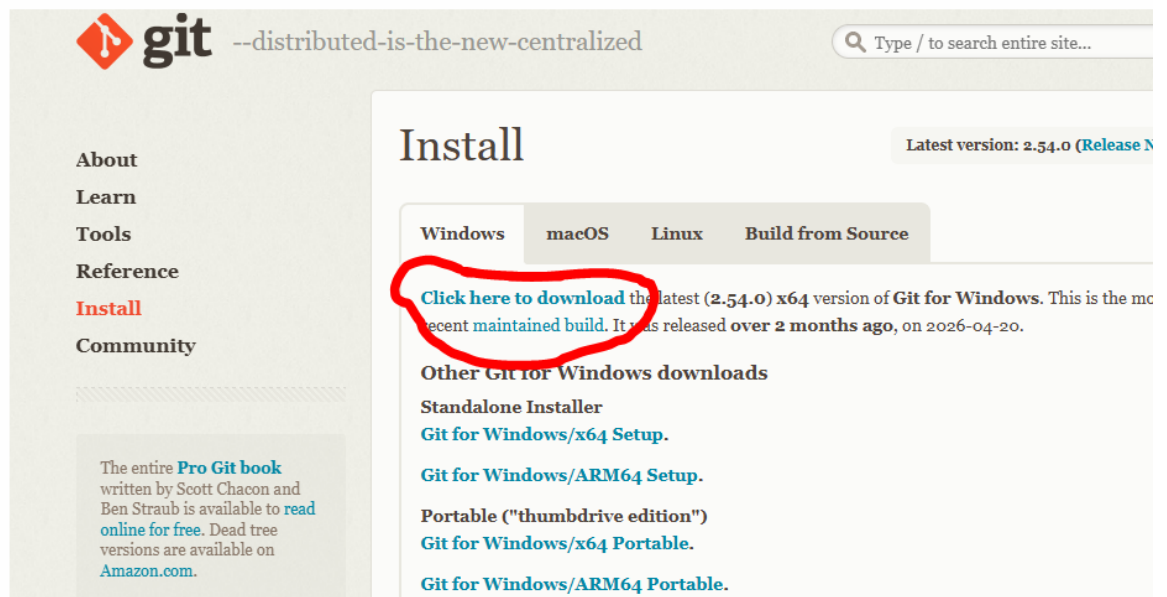
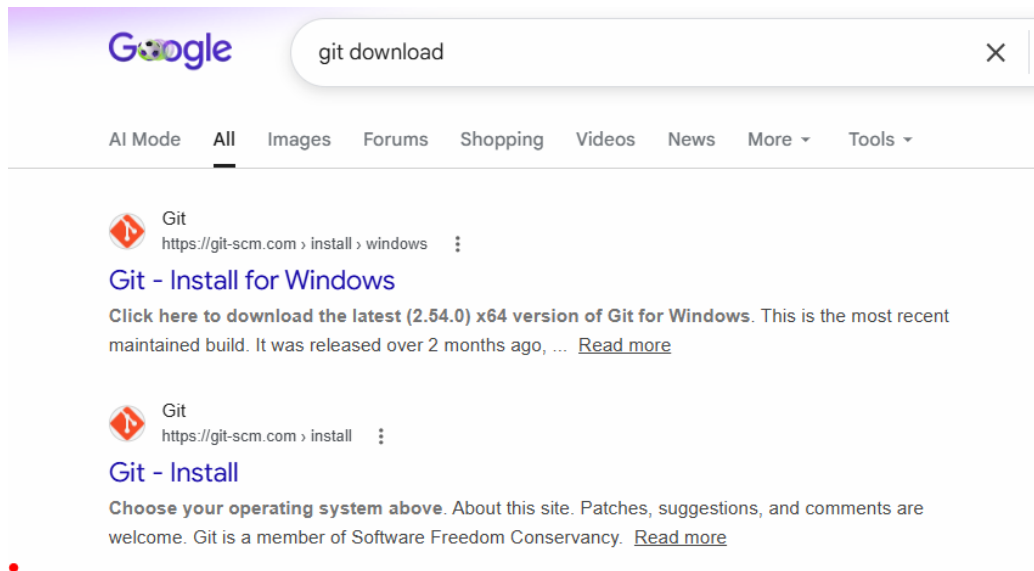
- **Accident Protection:** Because Git tracks every saved modification, you can view the full history and roll back your project to any previous commit if something breaks.
 - **Seamless Collaboration:** Multiple developers can work on the same project simultaneously. Git automatically handles merging changes and highlights conflicting edits for manual resolution.
 - **Offline Capability:** Because your own computer houses a complete local repository, you can commit, view history, and create branches entirely offline
- While Git is a command-line tool, it is frequently paired with cloud-based hosting platforms like GitHub or GitLab to store code remotely and streamline team workflows

Basic requirements to do the **exploring git commands through collaborative coding exercise**

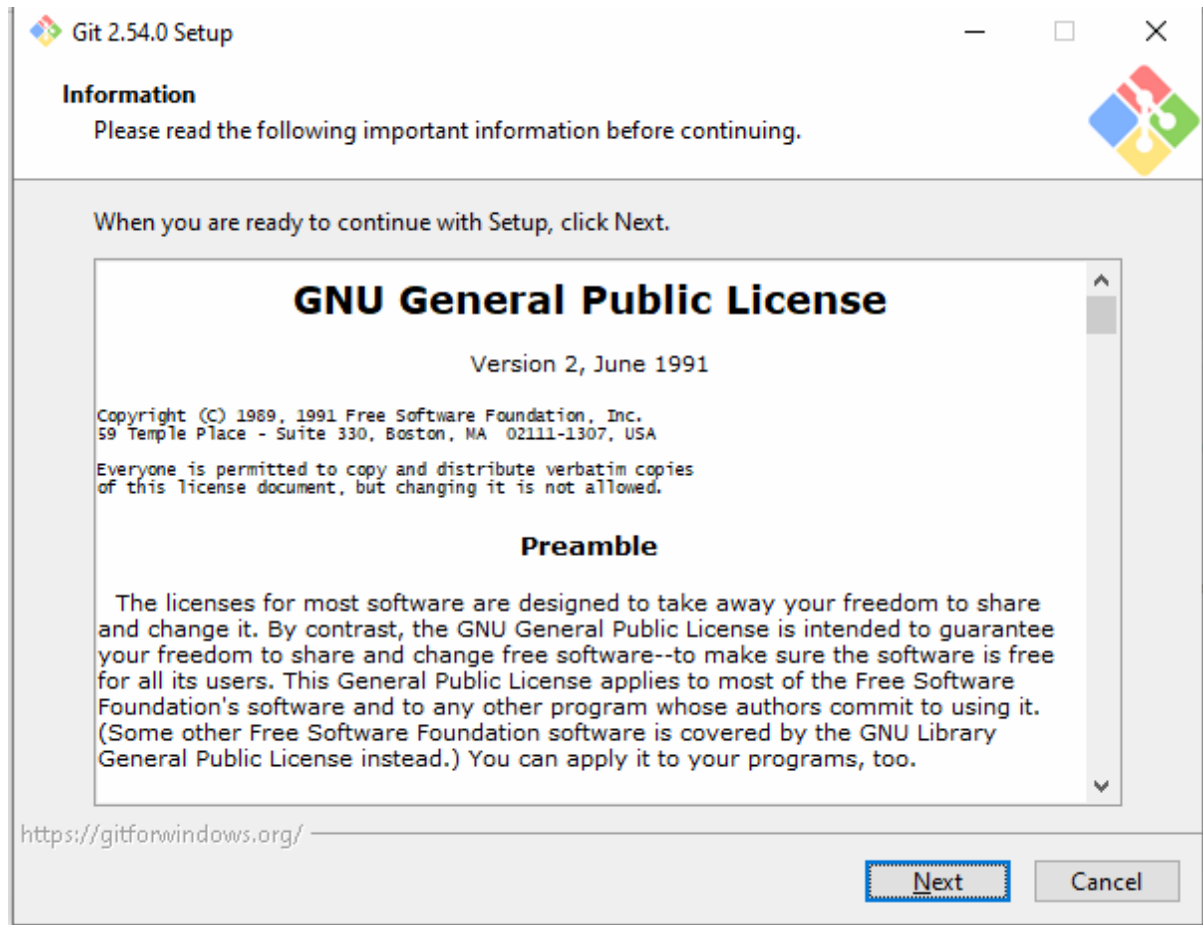
1. **Installing git to create and use the git command locally**
2. **Login to github to create your repository remotely**

Steps

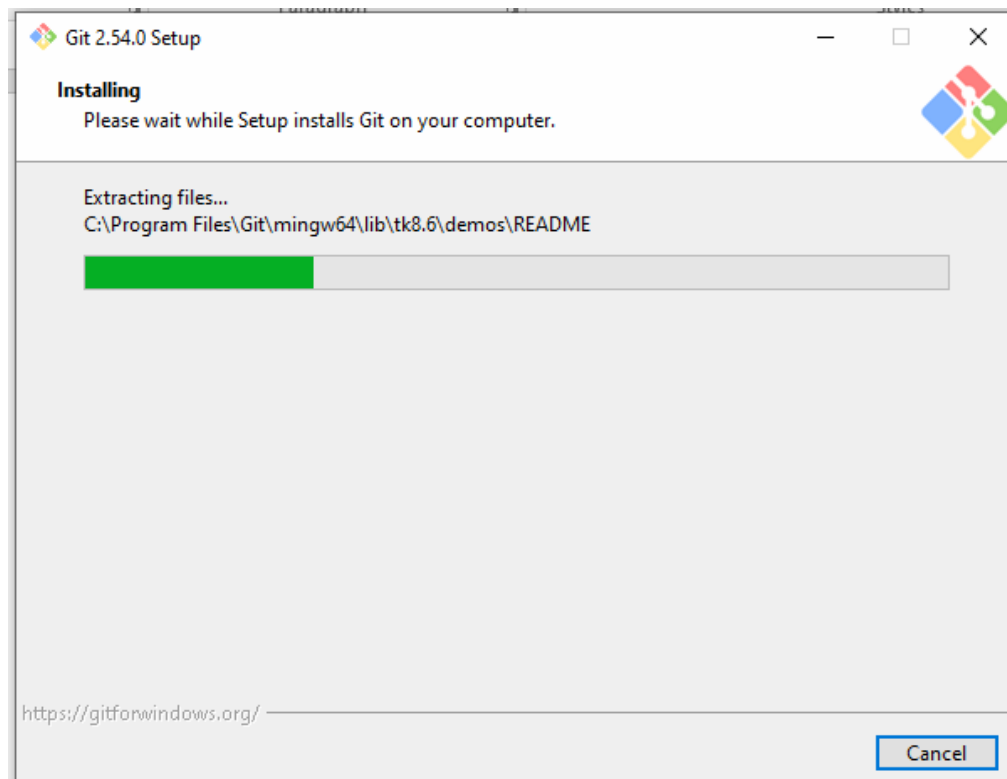
1. Download git software and install it



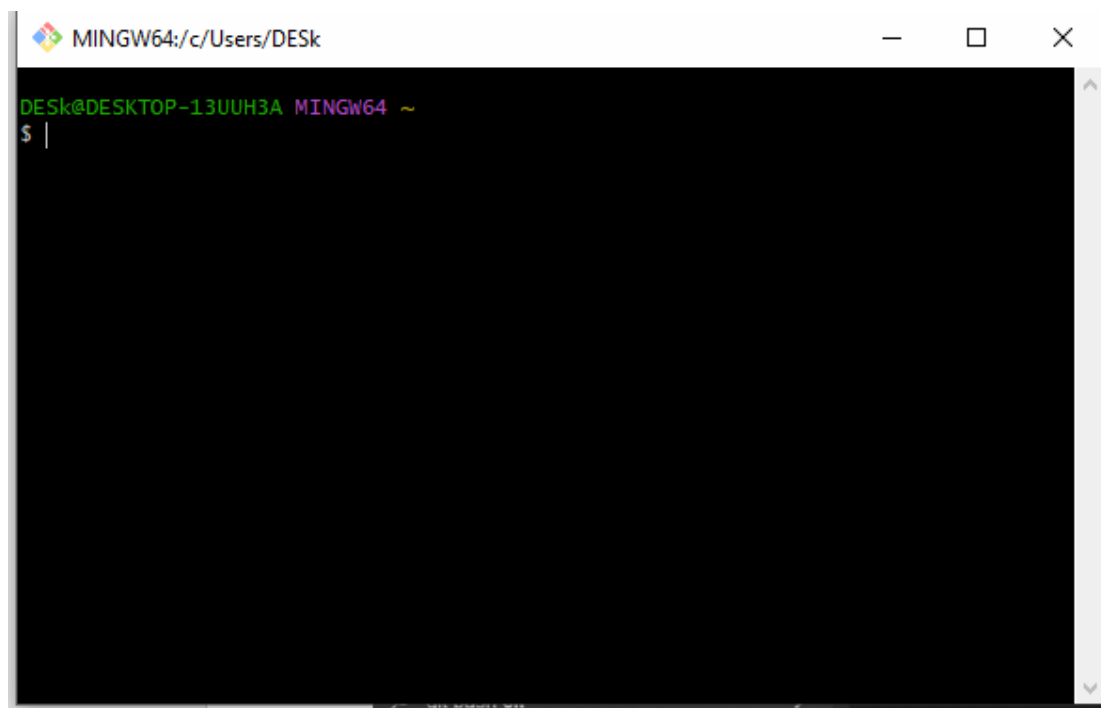
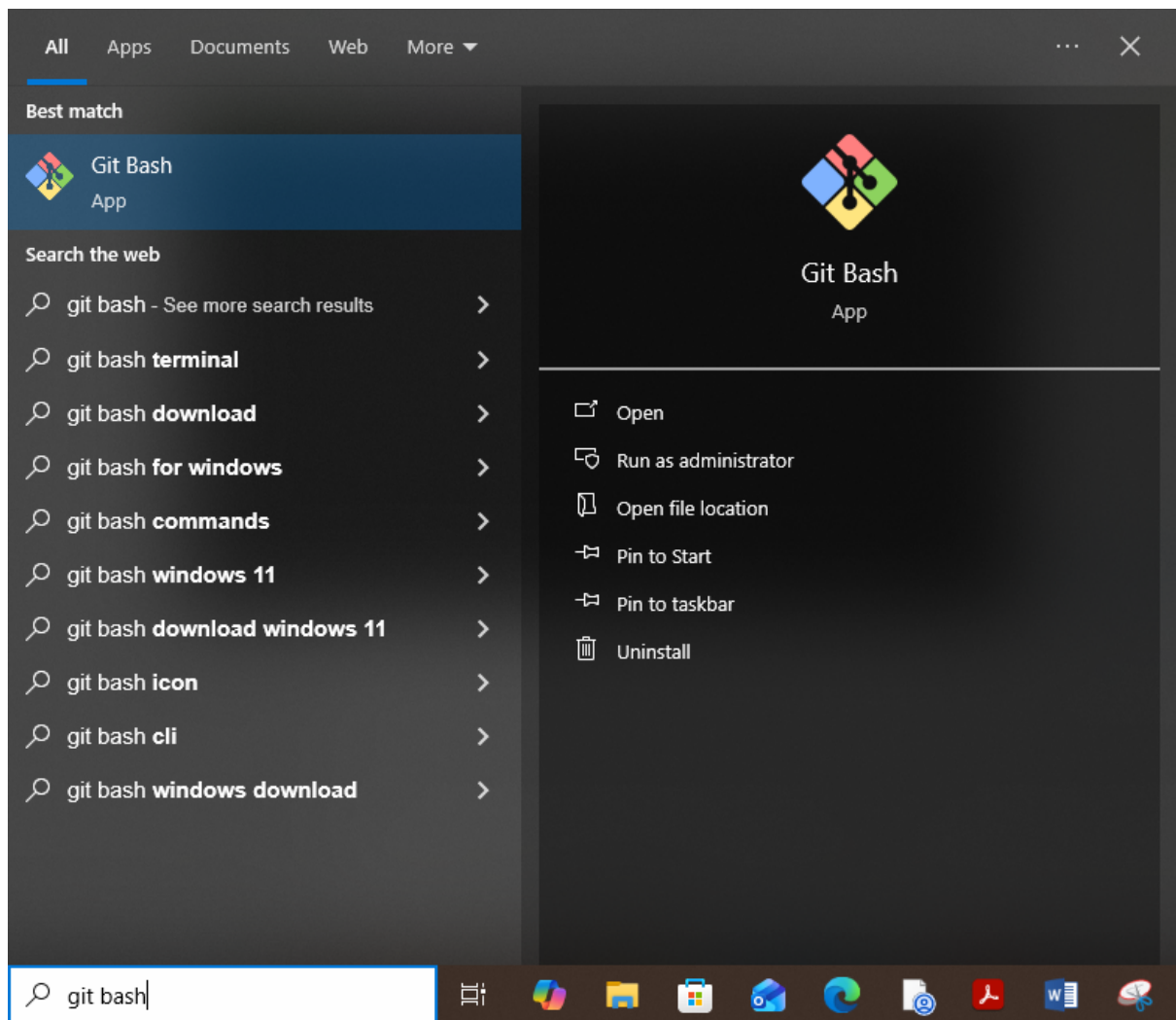
2. Setup git 2.54 as follows



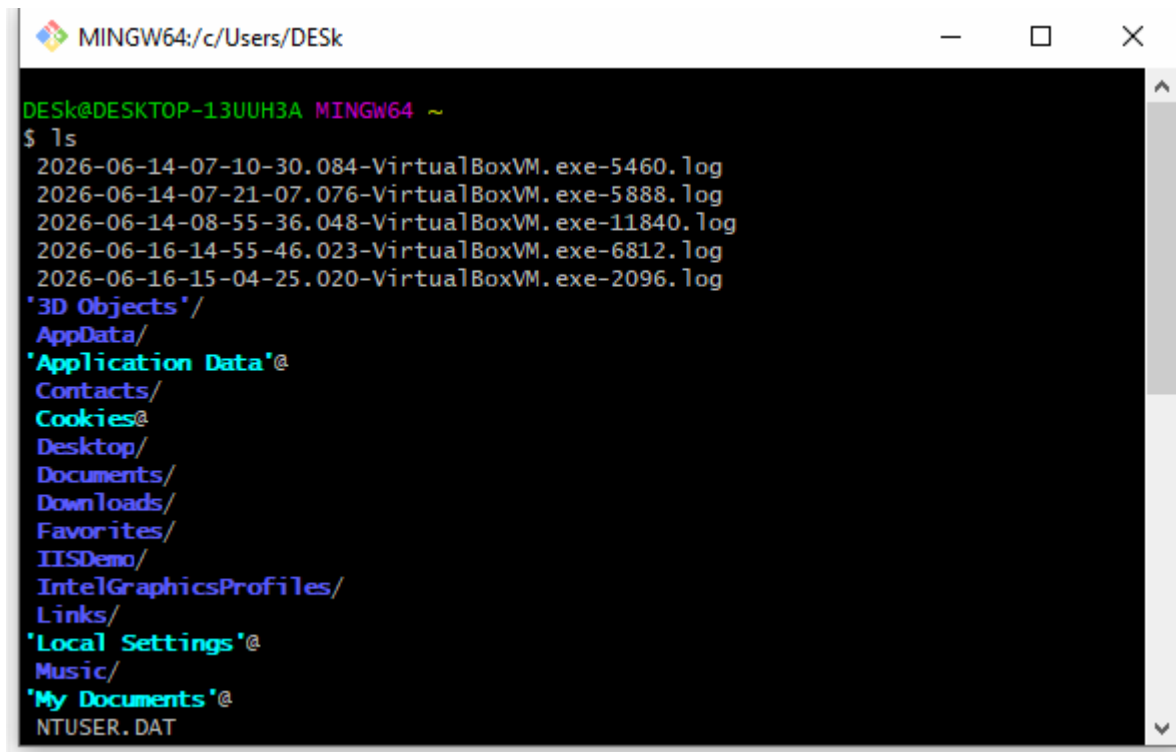
Click next to install no other configuration to modify



3. Once installed find git bash application or it automatically open.



ls command is used to list the directories and file in the current user

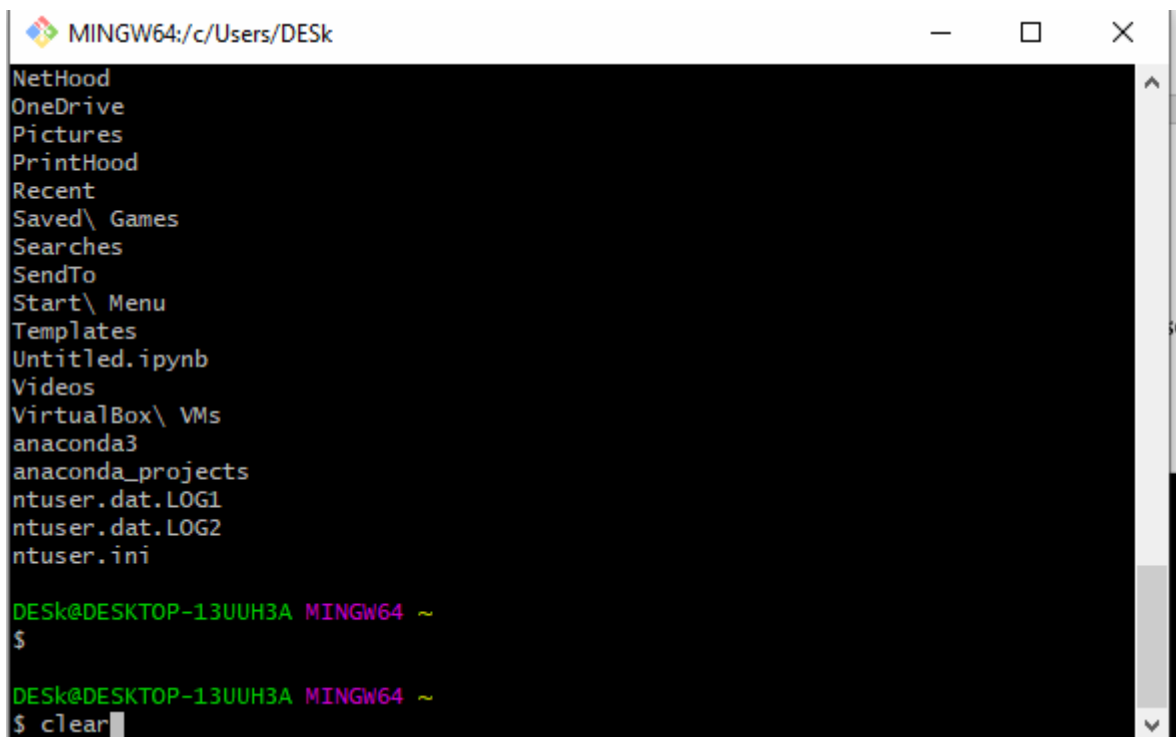


```

MINGW64:/c/Users/DESk
DESk@DESKTOP-13UUH3A MINGW64 ~
$ ls
2026-06-14-07-10-30.084-VirtualBoxVM.exe-5460.log
2026-06-14-07-21-07.076-VirtualBoxVM.exe-5888.log
2026-06-14-08-55-36.048-VirtualBoxVM.exe-11840.log
2026-06-16-14-55-46.023-VirtualBoxVM.exe-6812.log
2026-06-16-15-04-25.020-VirtualBoxVM.exe-2096.log
'3D Objects'/'
AppData/'
'Application Data'@
Contacts/'
Cookies@
Desktop/'
Documents/'
Downloads/'
Favorites/'
ITSDemo/'
IntelGraphicsProfiles/'
Links/'
'Local Settings'@
Music/'
'My Documents'@
NTUSER.DAT

```

Clear command will clear the current window



```

MINGW64:/c/Users/DESk
NetHood
OneDrive
Pictures
PrintHood
Recent
Saved\ Games
Searches
SendTo
Start\ Menu
Templates
Untitled.ipynb
Videos
VirtualBox\ VMs
anaconda3
anaconda_projects
ntuser.dat.LOG1
ntuser.dat.LOG2
ntuser.ini

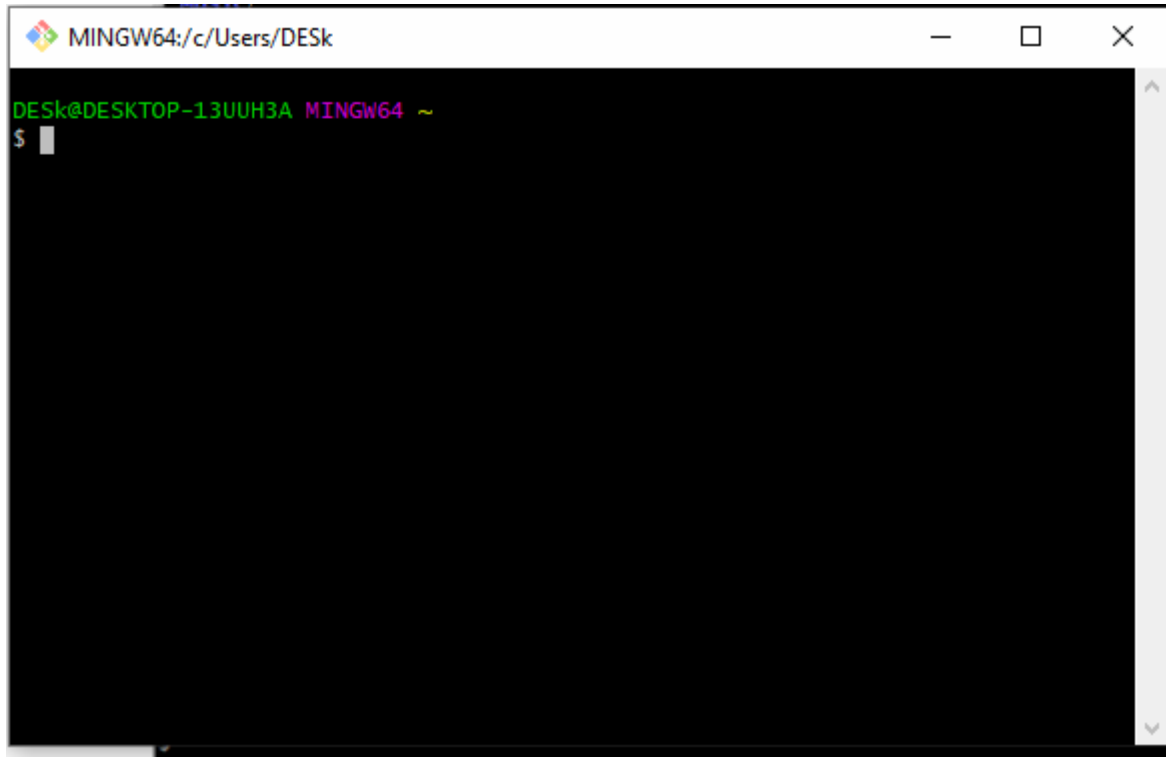
DESk@DESKTOP-13UUH3A MINGW64 ~
$

DESk@DESKTOP-13UUH3A MINGW64 ~
$ clear

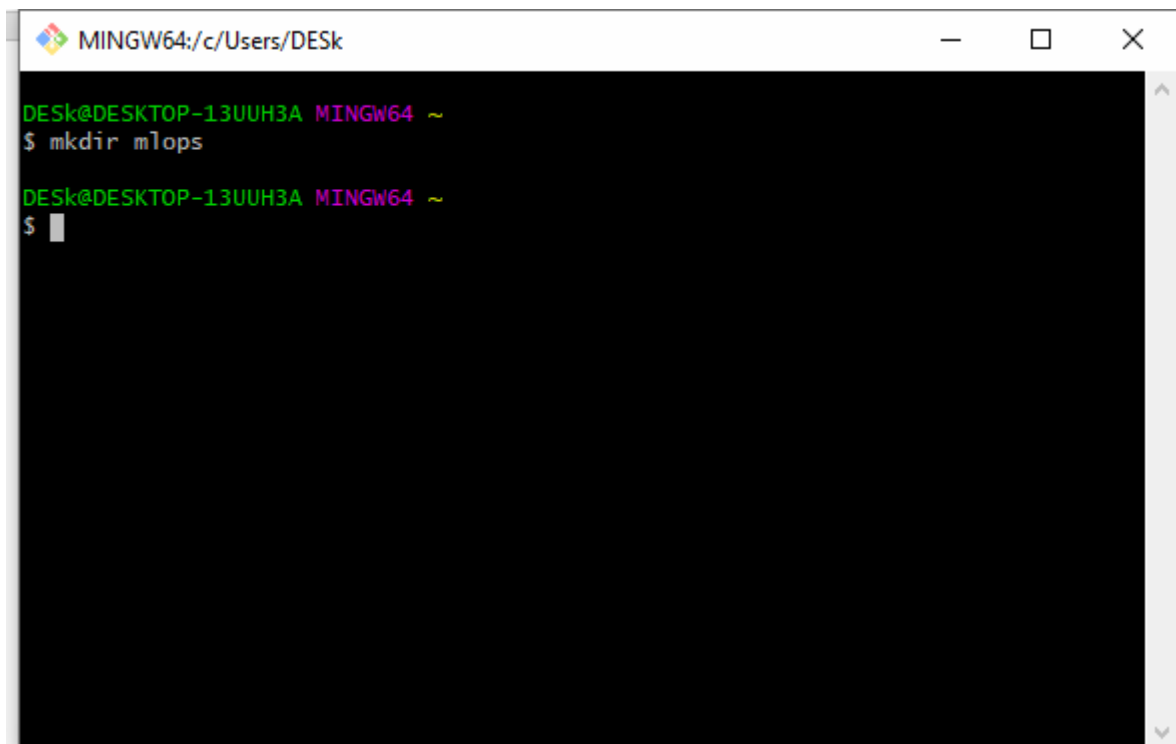
```

After clear command the windows like as follows

MLOPS LAB

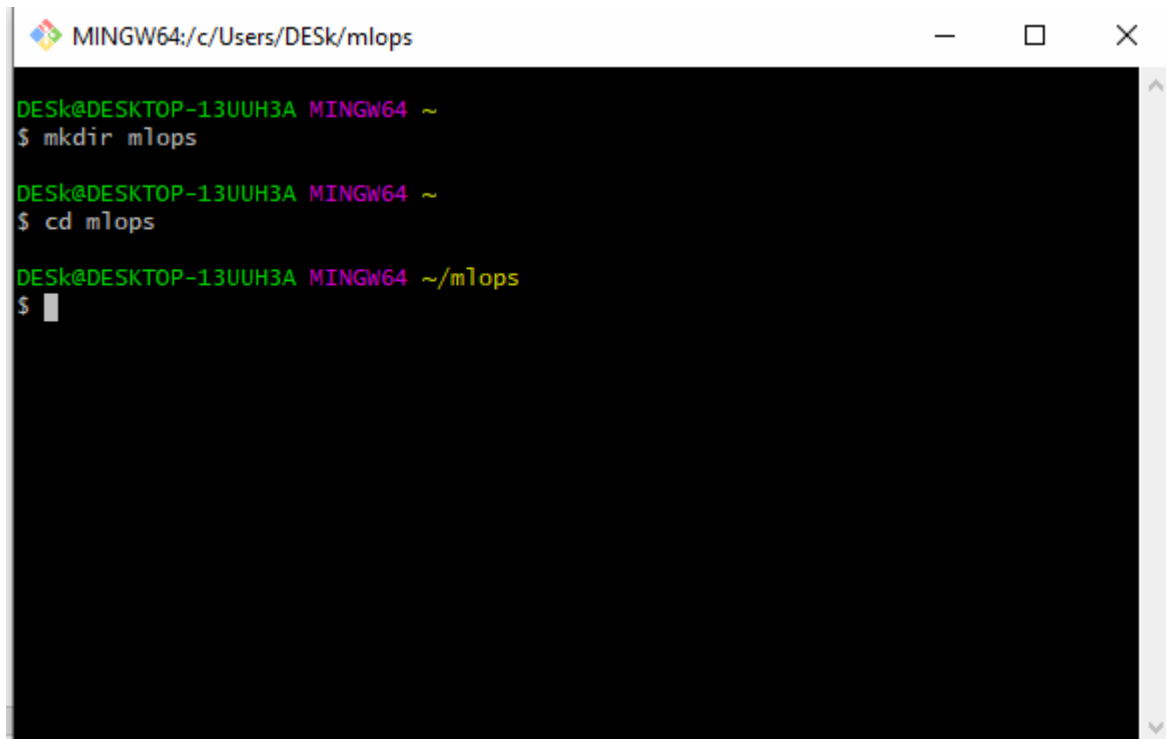


Create a new directory using mkdir
mkdir mlops



Get inside a particular dir

MLOPS LAB



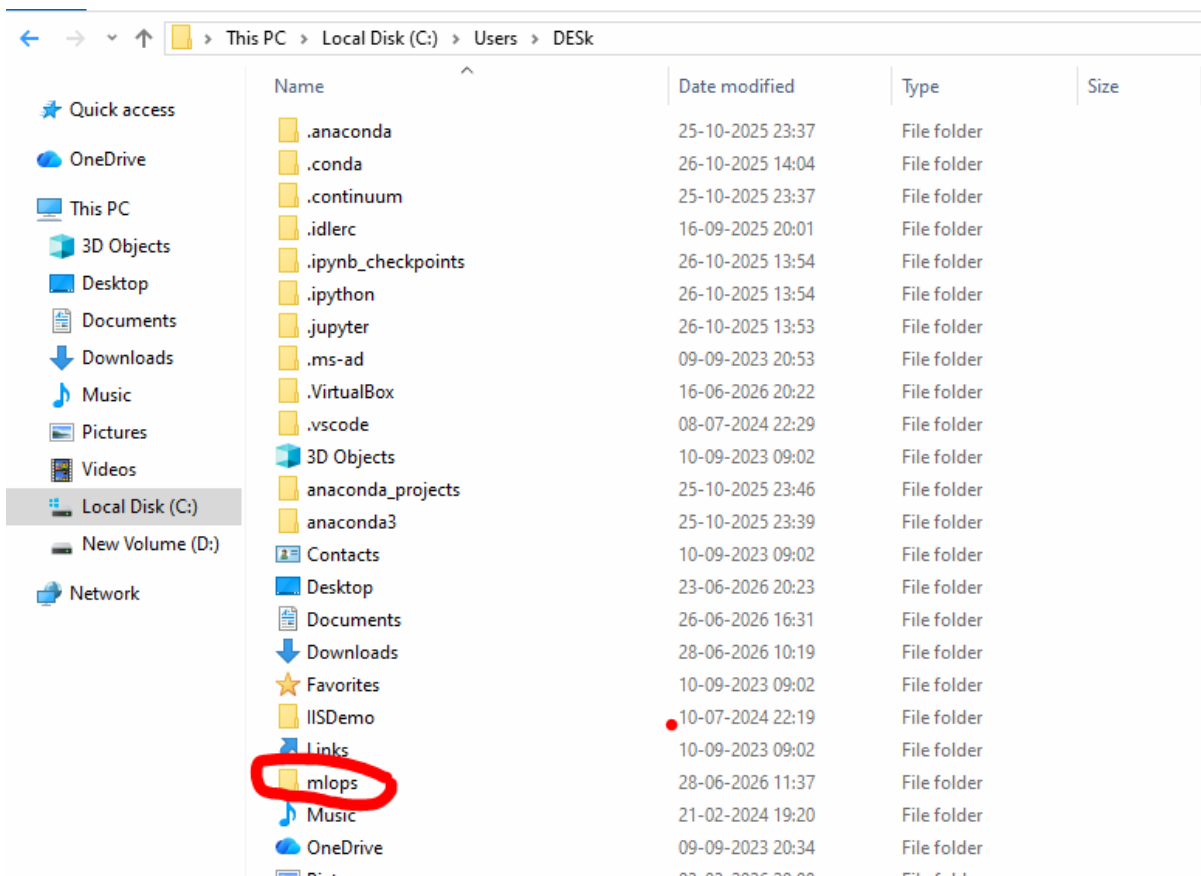
```
MINGW64:/c/Users/DESk/mlops

DESk@DESKTOP-13UUH3A MINGW64 ~
$ mkdir mlops

DESk@DESKTOP-13UUH3A MINGW64 ~
$ cd mlops

DESk@DESKTOP-13UUH3A MINGW64 ~/mlops
$
```

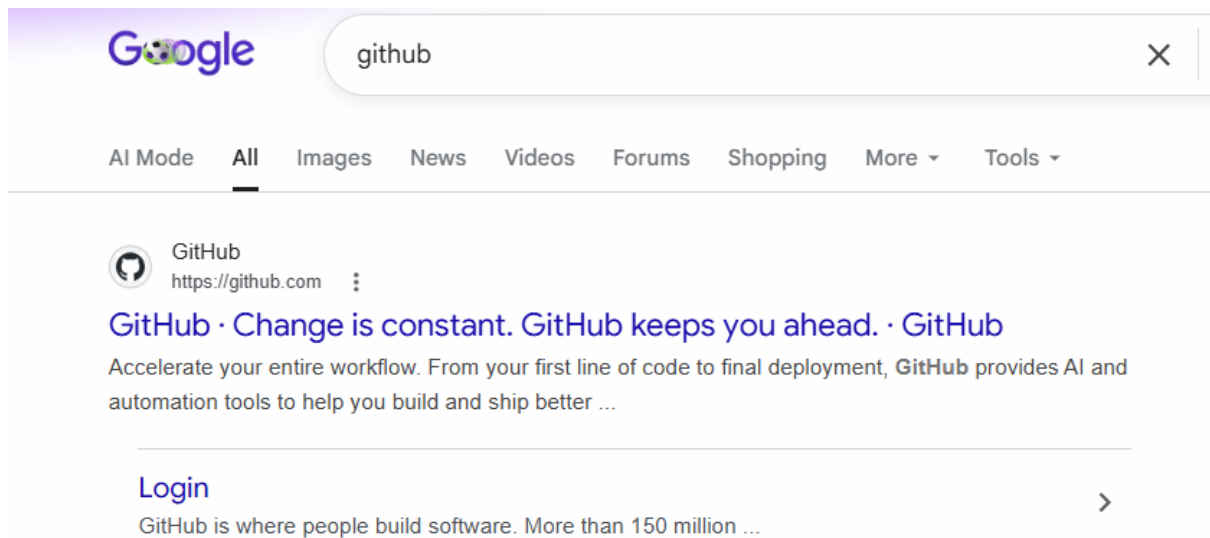
Here mlops directory. The created directory is residing in the following path, but this path may change depending upon the systems



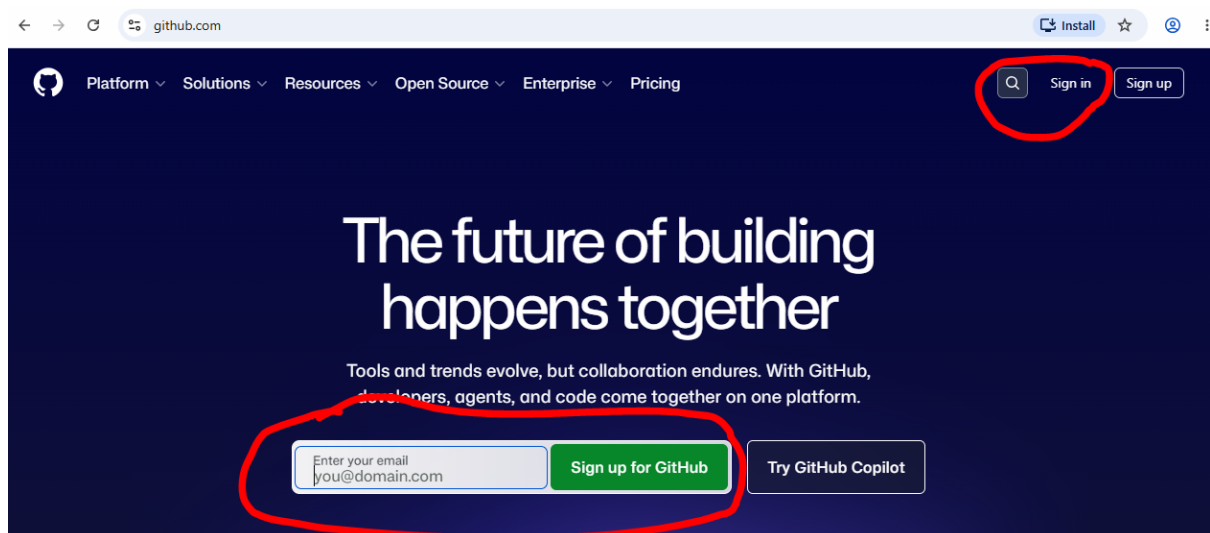
Login to github account

If you don't have github account please create it as follows

Go to github website and sign up with all your data



If you have already have an account simply sign in with your credentials



Else please sign up with your email id

You can sign up with your gmail id also

← → ↻ github.com/login



Sign in to GitHub

Username or email address

Password [Forgot password?](#)

[Sign in](#)

or

 Continue with Google

 Continue with Apple

New to GitHub? [Create an account](#)

[Sign in with a passkey](#)

Once click you continue with google it will directive to choose you gmail account

 Sign in with Google



Choose an account

to continue to [GitHub](#)

 **Sivakumar K**
sivakumar.kaliyappan@rajalakshmi.edu.in

 **Sivakumar K**
Signed out

 Use another account

Before using this app, you can review GitHub's [Privacy Policy](#) and [Terms of Service](#).

Complete your verification process

Device verification



Email

We just sent your authentication code via email to `s*****@rajalakshmi.edu.in`. The code will expire at 12:53PM IST.

Device Verification Code

Verify

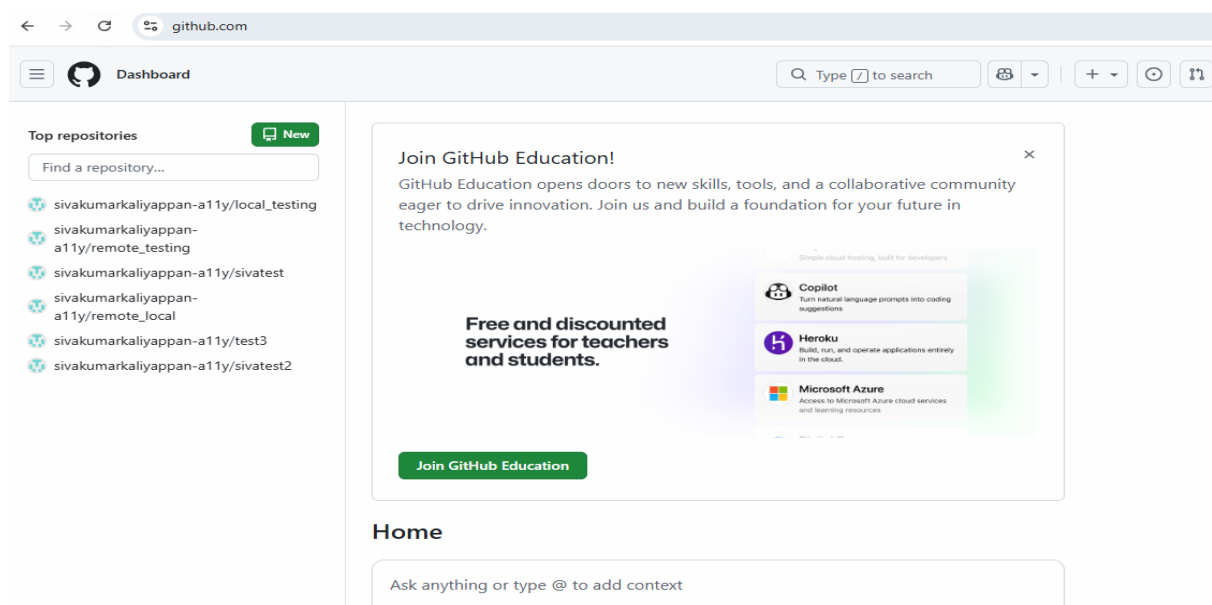
Having trouble verifying via email?

- [Re-send the authentication code](#)
- [Try GitHub Mobile for simplified device verification](#)

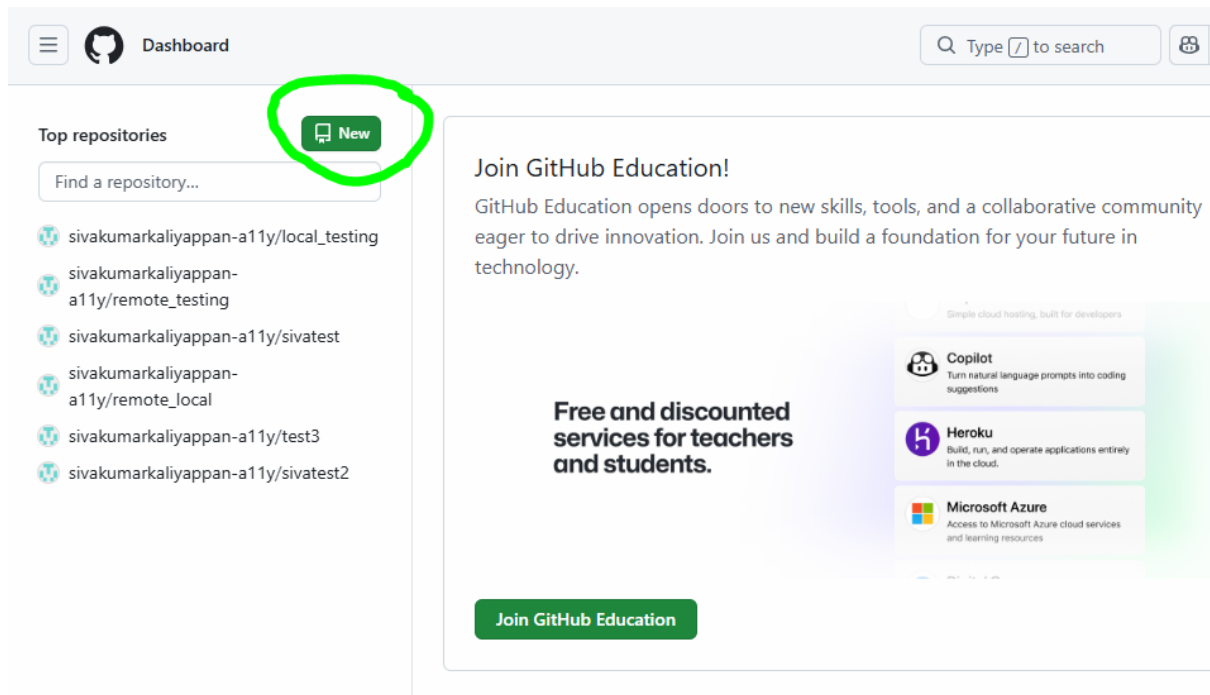
If you'd like to require verification on every

Once verification steps over then it will direct you to enter in to your github account

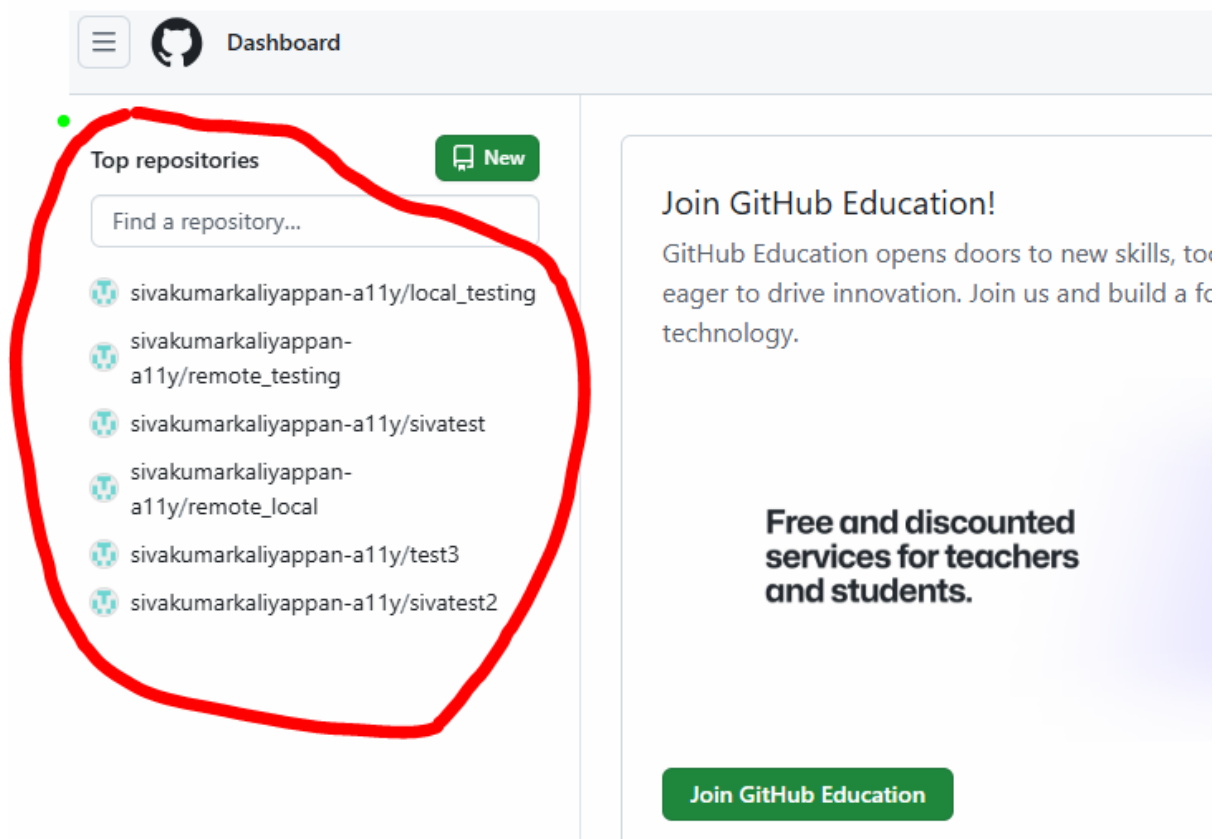
Or dashboard . the page as shown as follows



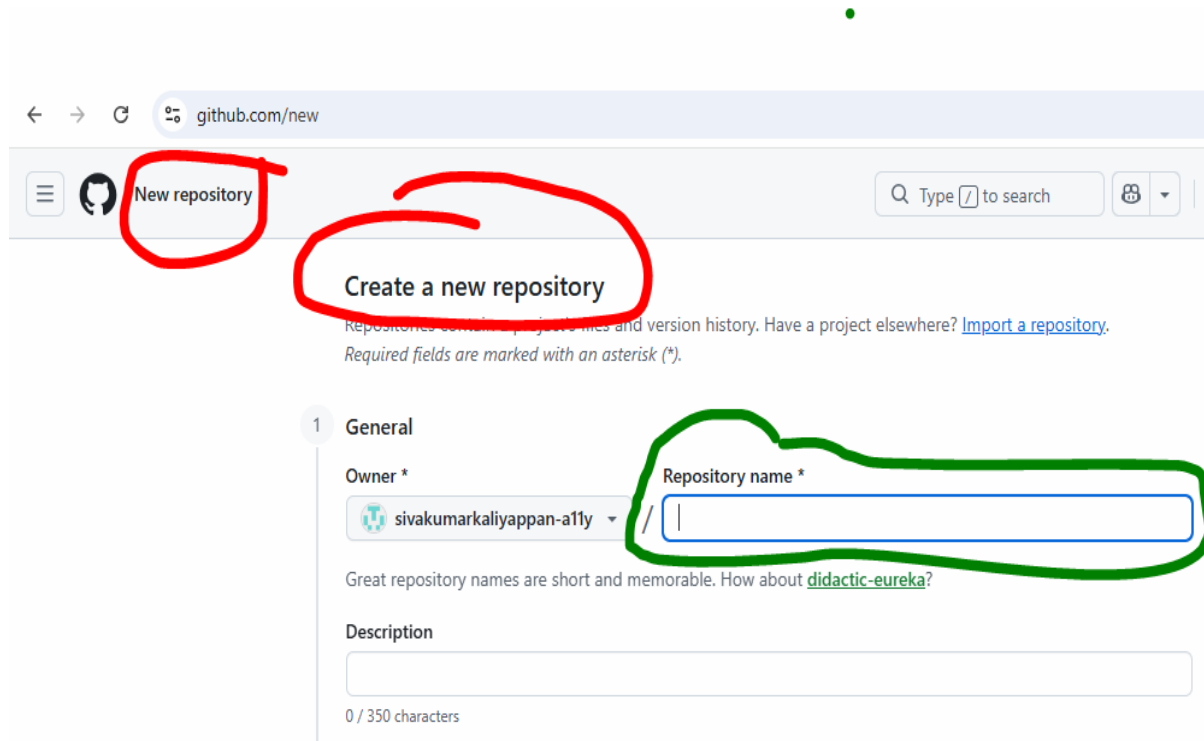
You can create a new repository as follows by clicking new button



Existing rtop repositories as follows



Creating new repository remotely as follows by clicking new button. Once you have clicked **new** button it will direct you as follows



The screenshot shows the GitHub 'Create a new repository' page. The browser address bar displays 'github.com/new'. In the top navigation bar, the 'New repository' button is circled in red. Below it, the heading 'Create a new repository' is also circled in red. The main form area is titled '1 General'. It contains two required fields, both marked with an asterisk (*): 'Owner *' and 'Repository name *'. The 'Owner *' field has a dropdown menu showing 'sivakumarkaliyappan-a11y'. The 'Repository name *' field is an empty text box, which is circled in green. Below these fields, there is a hint: 'Great repository names are short and memorable. How about [didactic-eureka?](#)'. At the bottom, there is a 'Description' text area and a character count '0 / 350 characters'.

Enter you new repository name here

MLOPS LAB

1 General

Owner *

 sivakumarkaliyappan-a11y ▾

Repository name *

new_repository_test

✓ new_repository_test is available.

Great repository names are short and memorable. How about [didactic-eureka?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

Create repository

Here we have created new repository name is called **new_repository_test**

Enter your repository purpose in the description box

1 General

Owner *  sivakumarkaliyappan-a11y / Repository name * new_repository_test

✔ new_repository_test is available.

Great repository names are short and memorable. How about [didactic-eureka?](#)

Description

it is for testing

18 / 350 characters

2 Configuration

Choose visibility * Choose who can see and commit to this repository  Public

✓  **Public**
Anyone on the internet can see this repository. You choose who can commit.

 **Private**
You choose who can see and commit to this repository.

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license
Licenses explain how others can use your code. [About licenses](#) No license

Create repository

Select whether your repository to be accessible by the public or keep it is private by clicking choose visibility

1

General

Owner *

 sivakumarkaliyappan-a11y ▾

 /

Repository name *

✔ new_repository_test is available.

Great repository names are short and memorable. How about [didactic-eureka?](#)

Description

18 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

✔  Public

Anyone on the internet can see this repository. You choose who can commit.

 Private

You choose who can see and commit to this repository.

Create repository

You make it Add READ ME is on or off

1 General

Owner *

 sivakumarkaliyappan-a11y ▾

Repository name *

new_repository_test

✔ new_repository_test is available.

Great repository names are short and memorable. How about [didactic-eureka?](#)

Description

it is for testing

18 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)On ☒

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

Create repository

You can choose which language or workspace you are going to work remotely by clicking **add .gitignore** and select the work environment

MLOPS LAB

github.com/new

1 **General**

Owner *  sivakumarkaliyappan-a11y / new_repository

Repository name * new_repository

Great repository names are short and memorable. How about...

Description

it is for testing

18 / 350 characters

2 **Configuration**

Choose visibility *
Choose who can see and commit to this repository

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

Add license
Licenses explain how others can use your code. [About licenses](#)

Choose a .gitignore template

q d

- ✓ No .gitignore
- Actionscript
- AppceleratorTitanium
- ArchLinuxPackages
- C
- C++
- CFWheels
- CMake
- CUDA
- CakePHP
- ChefCookbook

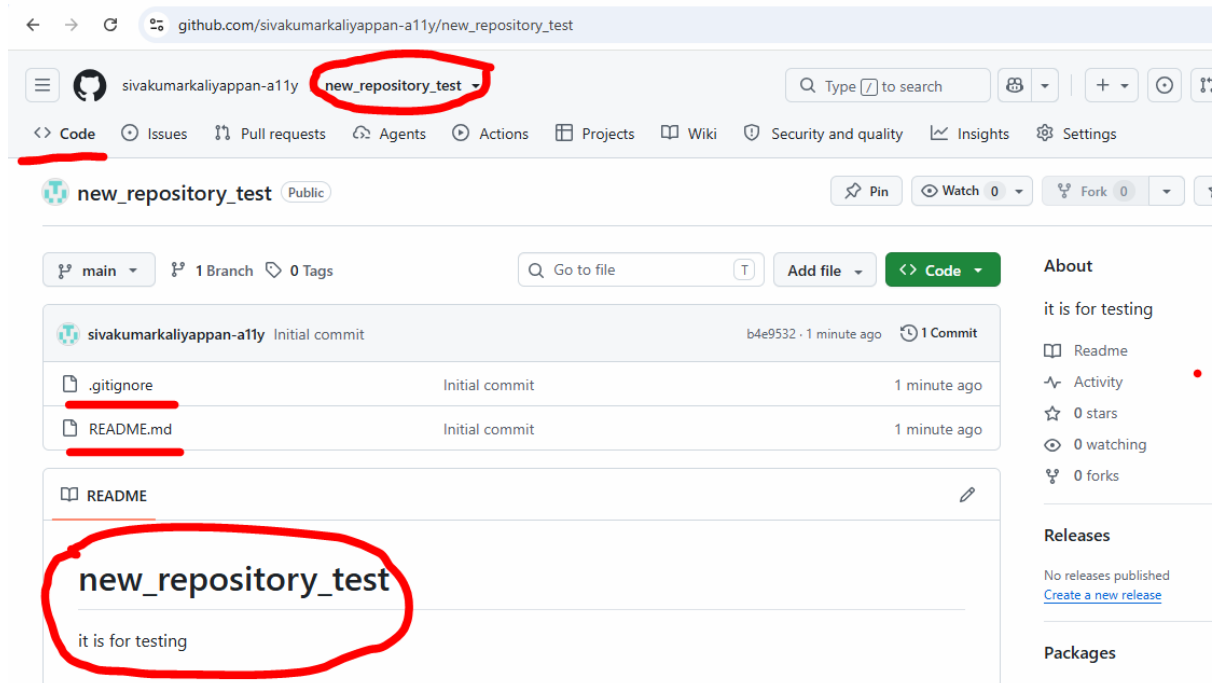
No .gitignore

No license

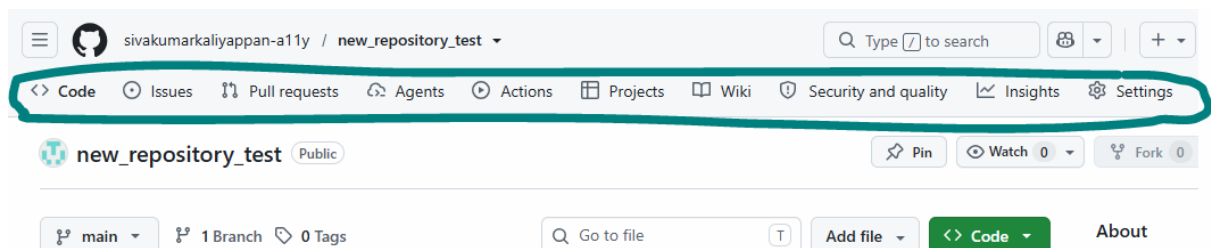
Create repository

After selecting your workspace or language click **create repository**

Once you have clicked **create repository** then it will display your initial windows as follows



Here there are some tab is available for some purpose. This will see later



When you click the code it will display two tab

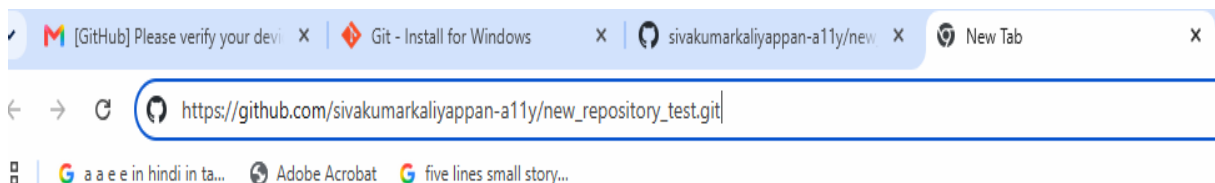
Local and codespace

In local we have HTTPS, SSH, GitHub CLI

In HTTPS it show address where your repository created

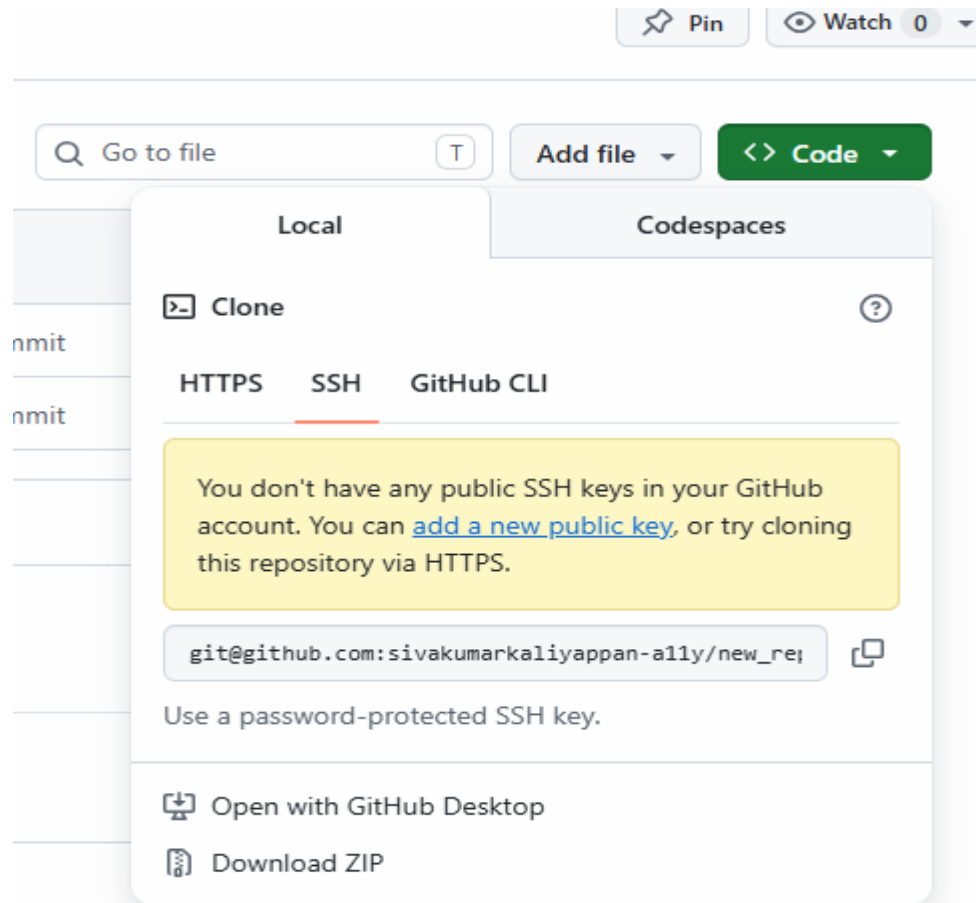
https://github.com/sivakumarkaliyappan-a11y/new_repository_test.git

by copy the above URL and past in the addressbar it open you repository if you are already open else it will ask user name and password

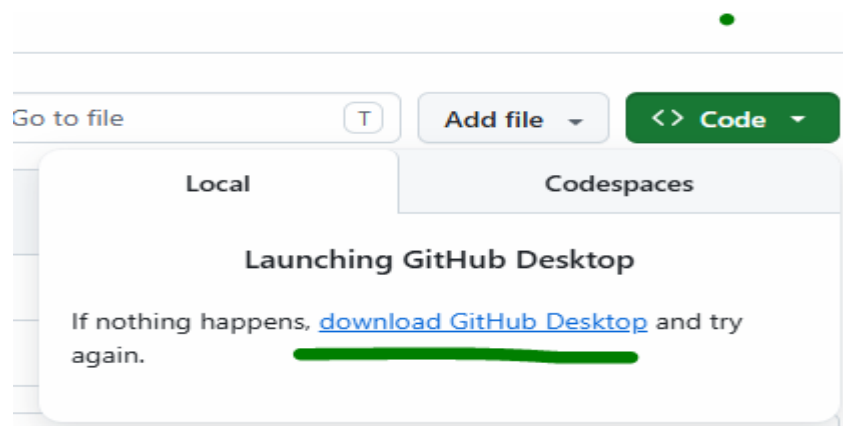


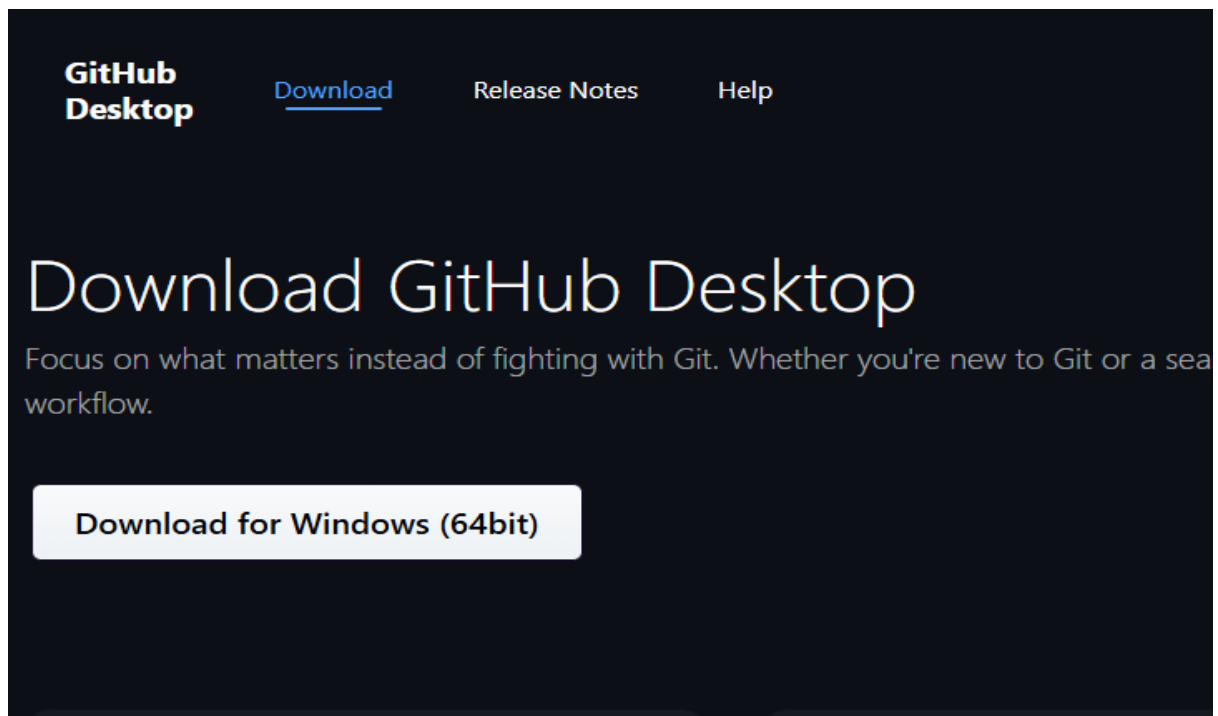
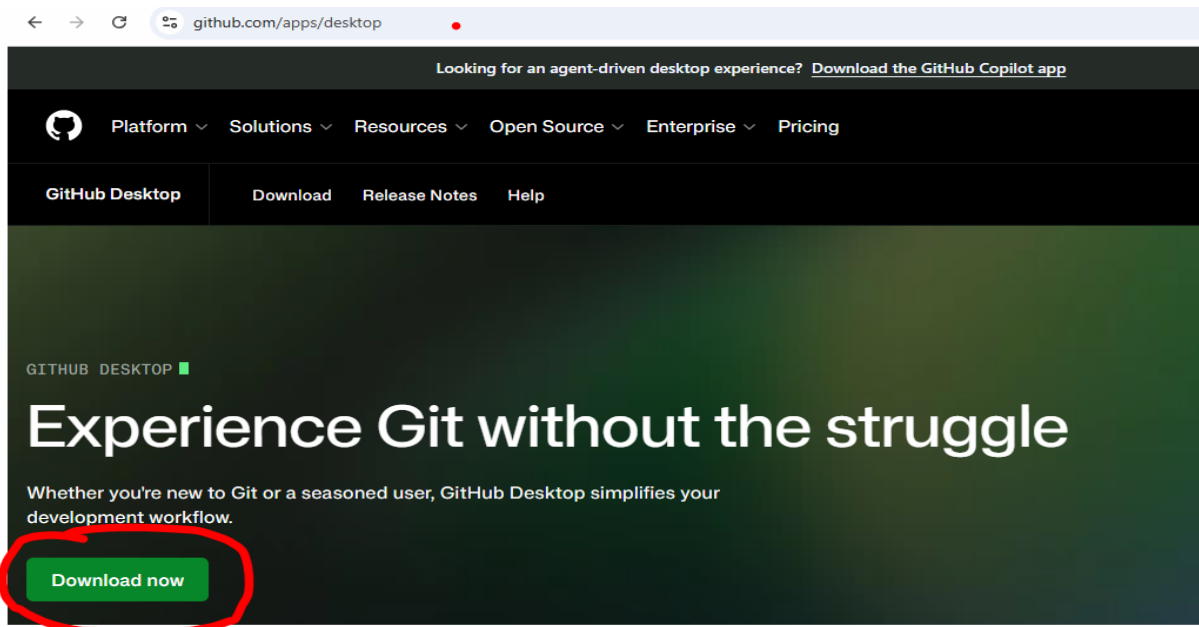
In SSH you can login your repository through SSH by using user name and password

[git@github.com:sivakumarkaliyappan-a11y/new_repository_test.git](https://github.com/sivakumarkaliyappan-a11y/new_repository_test.git)



When you click open with GitHub Desktop it will direct you to download github desktop version software as follows . if you want you can install desktop version

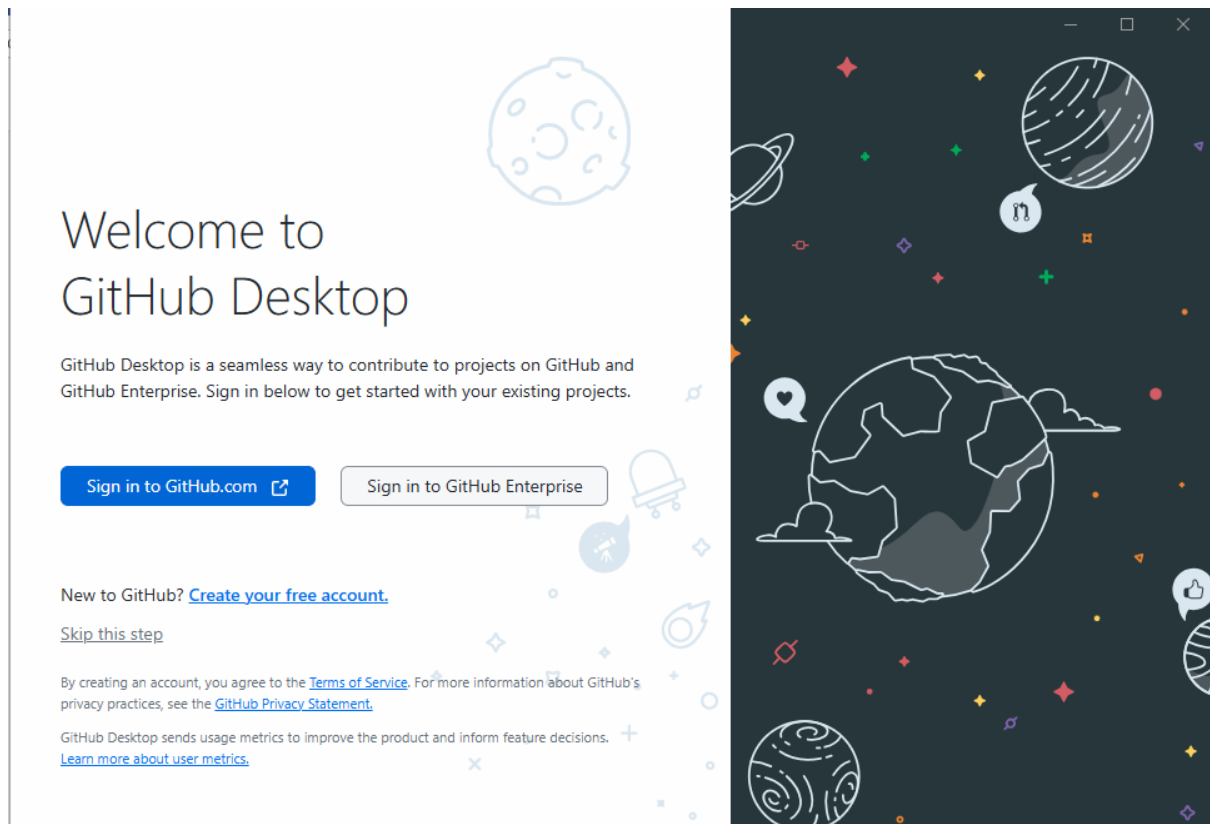




Once downloaded then install it



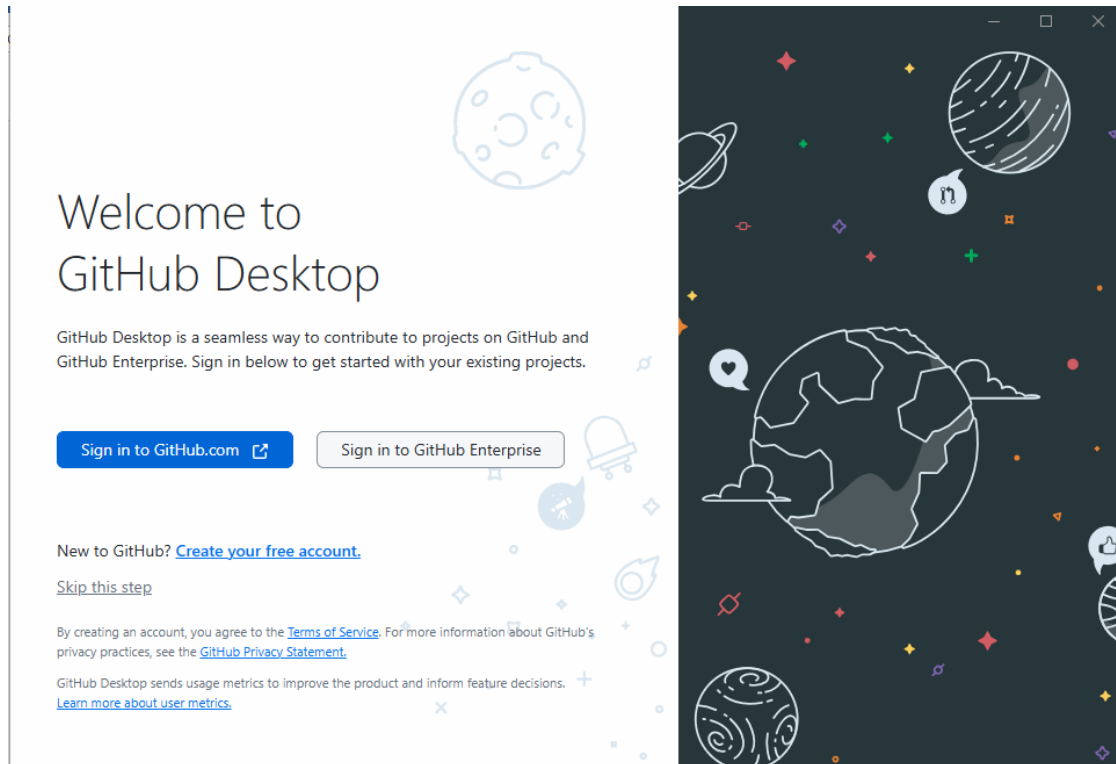
Once installed the desk top version window is appear like this



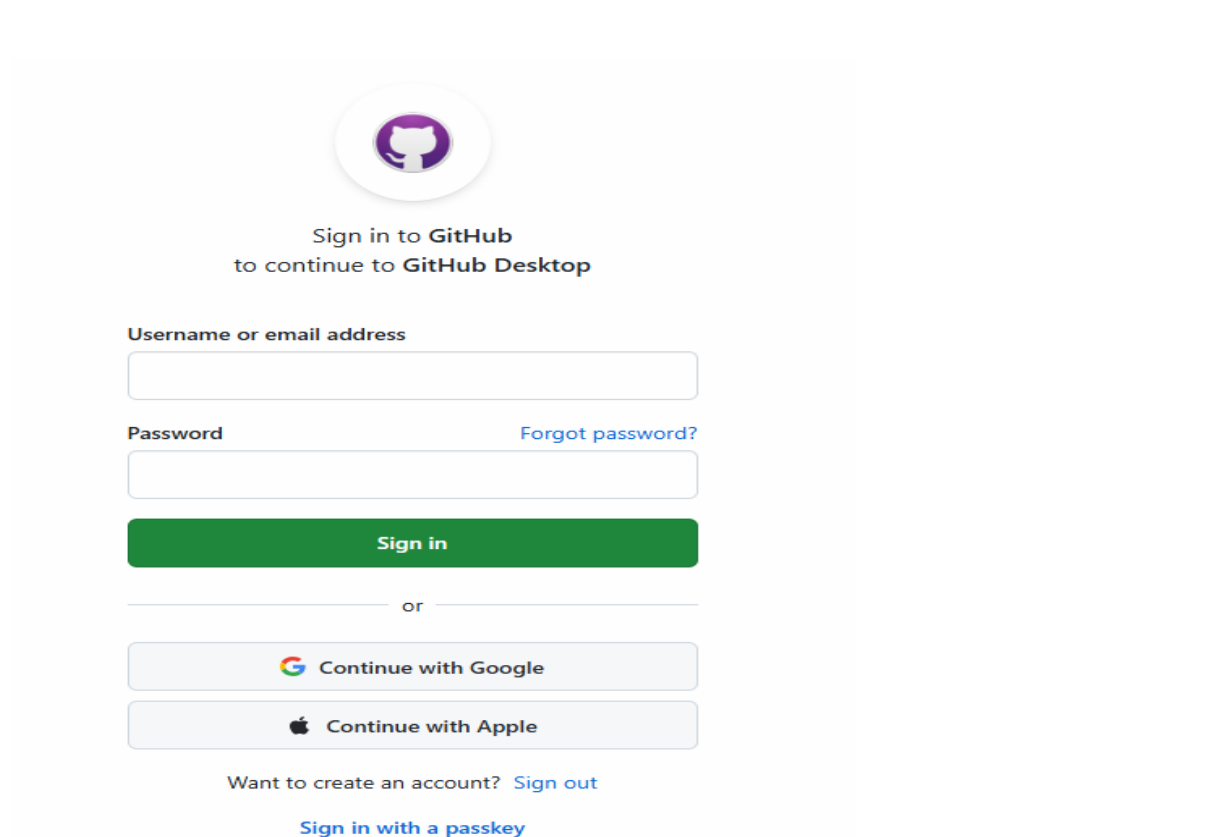
Ex No 2 Exploring you github desktop version

Open github desktop version software

If you have an account, click [sign in github.com](#)



Once clicked it will direct you to login page



The image shows the GitHub Desktop login interface. At the top, there is a GitHub logo inside a circle. Below it, the text reads "Sign in to GitHub to continue to GitHub Desktop". There are two input fields: "Username or email address" and "Password". A "Forgot password?" link is next to the password field. A green "Sign in" button is below the fields. Below the button, there is a horizontal line with the word "or" in the center. Underneath, there are two buttons: "Continue with Google" and "Continue with Apple". At the bottom, there is a link "Want to create an account? Sign out" and a link "Sign in with a passkey".

Sign in to **GitHub**
to continue to **GitHub Desktop**

Username or email address

Password [Forgot password?](#)

Sign in

or

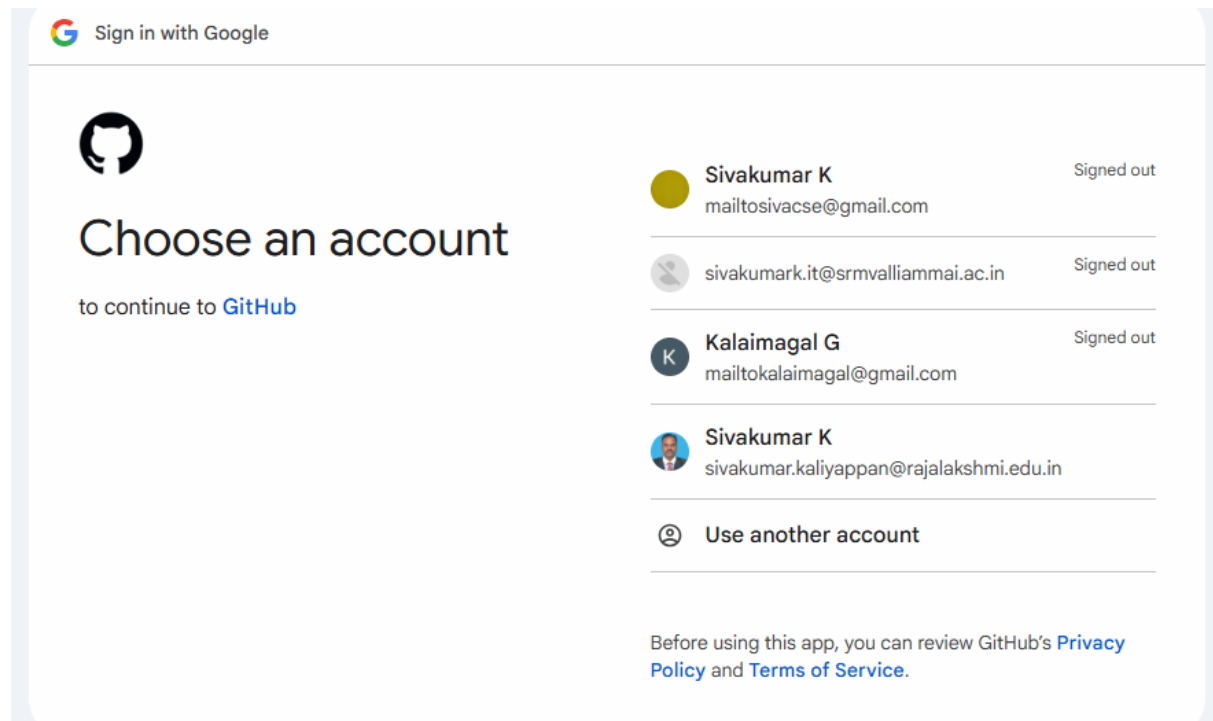
 **Continue with Google**

 **Continue with Apple**

Want to create an account? [Sign out](#)

[Sign in with a passkey](#)

The next page appear like this



The image shows the GitHub account selection screen. At the top, there is a "Sign in with Google" button. Below it, there is a GitHub logo and the text "Choose an account to continue to GitHub". On the right side, there is a list of accounts. Each account entry includes a profile picture, the name, the email address, and the status "Signed out". At the bottom, there is a link "Before using this app, you can review GitHub's Privacy Policy and Terms of Service.".

 **Sign in with Google**



Choose an account
to continue to **GitHub**

	Sivakumar K mailto:sivacse@gmail.com	Signed out
	sivakumark.it@srmvalliammai.ac.in	Signed out
	Kalaimagal G mailto:kalaimagal@gmail.com	Signed out
	Sivakumar K sivakumar.kaliyappan@rajalakshmi.edu.in	
	Use another account	

Before using this app, you can review GitHub's [Privacy Policy](#) and [Terms of Service](#).

Once you have chosen your account it will open as follows if you have logged in yours google account

MLOPS LAB

Dashboard

Type ↵ to search

You're already signed in to this account.

Top repositories

New

Find a repository...

sivakumarkaliyappan-a11y/local_testing

sivakumarkaliyappan-a11y/remote_testing

sivakumarkaliyappan-a11y/sivatest

sivakumarkaliyappan-a11y/new_repository_test

sivakumarkaliyappan-a11y/remote_local

sivakumarkaliyappan-a11y/test3

sivakumarkaliyappan-a11y/sivatest2

Join GitHub Education!

GitHub Education opens doors to new skills, tools, and a collaborative community eager to drive innovation. Join us and build a foundation for your future in technology.

Free and discounted services for teachers and students.

Copilot

Turn natural language prompts into coding suggestions

Heroku

Build, run, and operate applications entirely in the cloud.

Microsoft Azure

Access to Microsoft Azure cloud services and learning resources

Join GitHub Education

Home

To master Git for collaboration, you need to understand how it tracks files locally and syncs them with a remote team. **The core collaborative workflow relies on branching to isolate features, fetching to stay updated, and pulling or pushing to share code.** This breakdown covers a step-by-step collaborative coding simulation

Collaborative Development with Git

In today's fast-paced software development environment, collaboration among developers is essential. Git, a powerful version control system, facilitates this collaboration by enabling multiple developers to work simultaneously on a codebase without stepping on each other's toes. In this article, we will explore the principles and best practices of collaborative development using Git, empowering you to make your team's workflow more efficient and organized.

What is Git?

Git is a distributed version control system (VCS) created by Linus Torvalds in 2005. Its primary purpose is to manage source code changes in a collaborative setting. Unlike traditional version control systems, Git allows every developer to have a complete local copy of the repository, making it faster and more efficient.

Why Use Git for Collaborative Development?

Here are some reasons why Git is the preferred choice for collaborative development:

- **Branching and Merging:** Git allows developers to create branches for new features, bug fixes, or experiments, which they can later merge into the main codebase.
- **Version History:** Git records every change made to the code, enabling developers to track modifications and revert to previous versions if necessary.
- **Collaboration:** Multiple developers can work on the same project seamlessly, with Git managing any conflicts that may arise during integration.
- **Distributed Nature:** Each developer has a full clone of the repository, allowing for offline work and enhancing security.

Setting Up a Collaborative Environment with Git

To begin collaborative development with Git, follow these steps to set up your environment:

1. Install Git

First, you need to install Git on your local machine. You can download it from git-scm.com. Follow the installation prompts for your respective operating system.

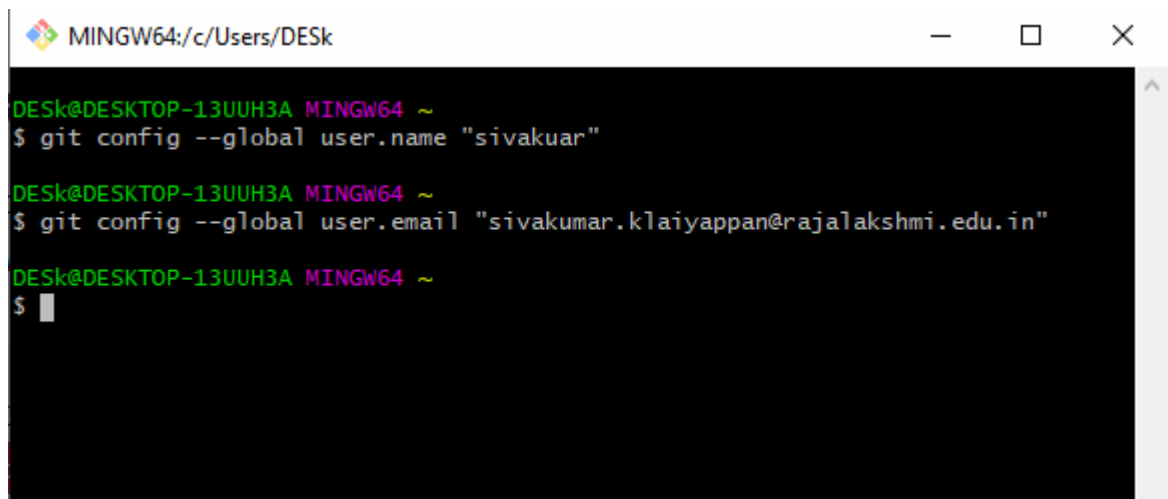
2. Configure Git

After installation, configure your Git environment by setting your username and email. This information will be recorded with every commit you make:

Enter the following command to setup your name and email id

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

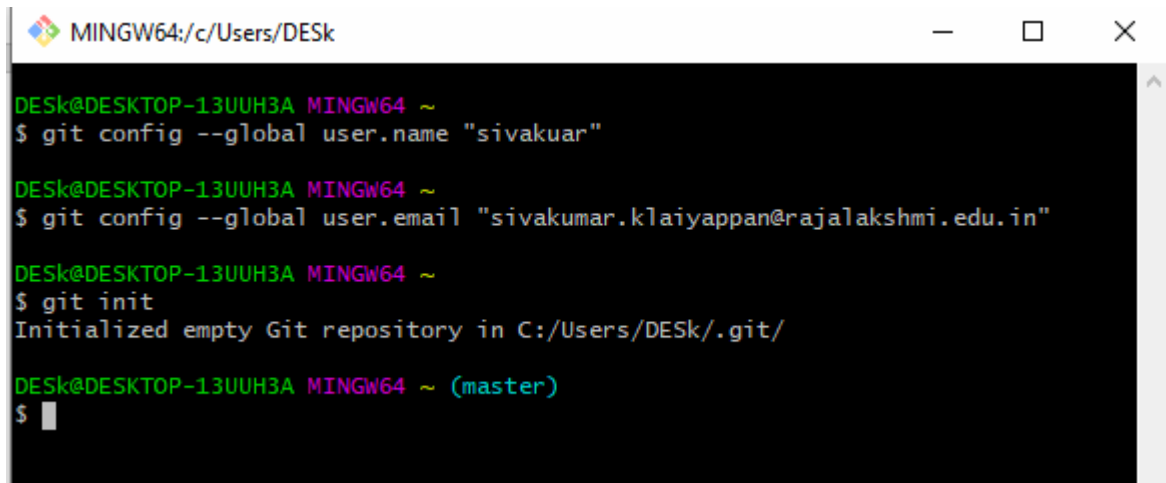
A screenshot of a Windows command prompt window titled "MINGW64:/c/Users/DESk". The window shows the execution of two Git configuration commands. The first command is "git config --global user.name 'sivakuar'", and the second is "git config --global user.email 'sivakumar.klaiyappan@rajalakshmi.edu.in'". The prompt is currently at the third line, showing "DESk@DESKTOP-13UUH3A MINGW64 ~" and "\$".

```
DESk@DESKTOP-13UUH3A MINGW64 ~  
$ git config --global user.name "sivakuar"  
  
DESk@DESKTOP-13UUH3A MINGW64 ~  
$ git config --global user.email "sivakumar.klaiyappan@rajalakshmi.edu.in"  
  
DESk@DESKTOP-13UUH3A MINGW64 ~  
$
```

3. Create a New Repository

To create a new Git repository, navigate to your project directory and use the following command:

Git init



```
MINGW64:/c/Users/DESk
DESk@DESKTOP-13UUH3A MINGW64 ~
$ git config --global user.name "sivakuar"

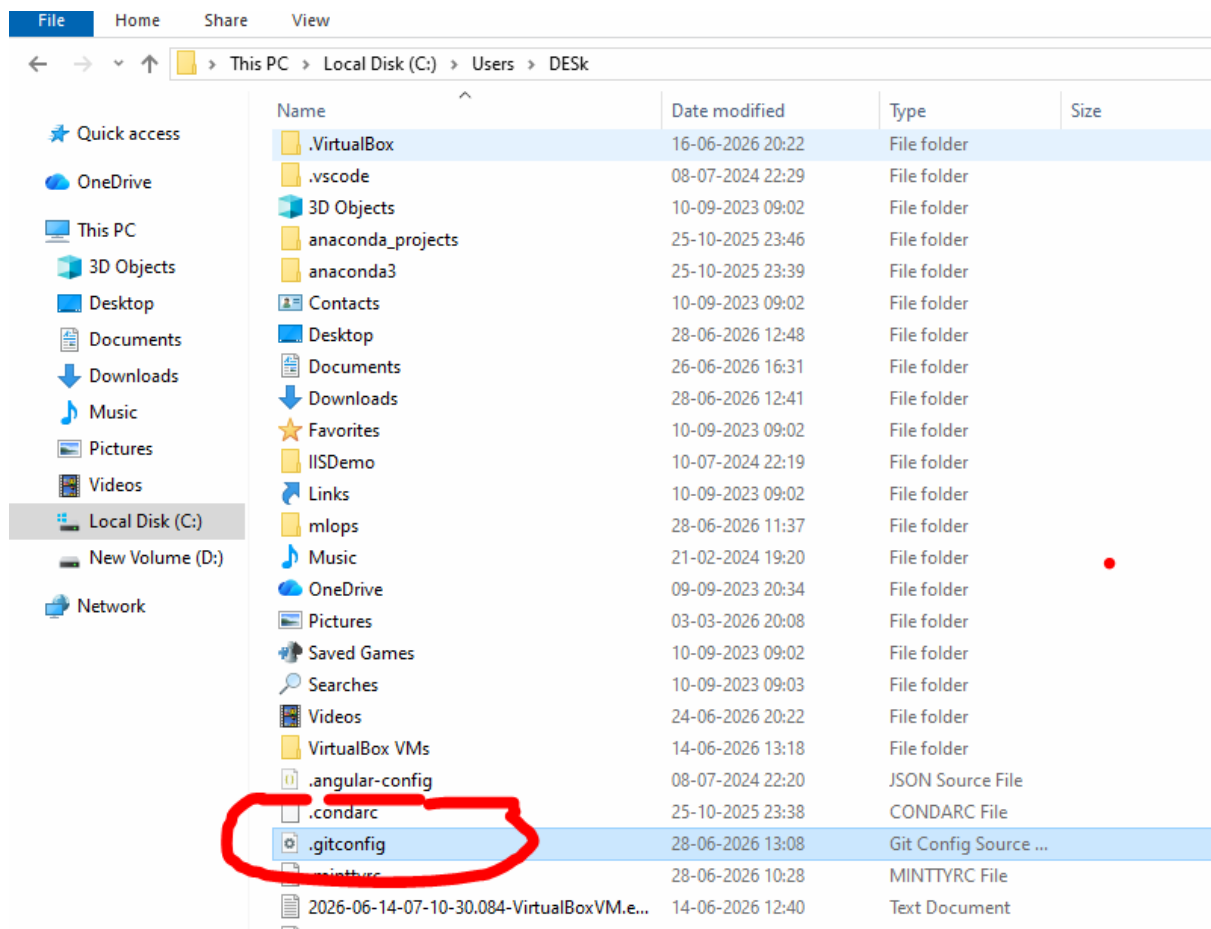
DESk@DESKTOP-13UUH3A MINGW64 ~
$ git config --global user.email "sivakumar.klaiyappan@rajalakshmi.edu.in"

DESk@DESKTOP-13UUH3A MINGW64 ~
$ git init
Initialized empty Git repository in C:/Users/DESk/.git/

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
```

Here no repository created no or folder created

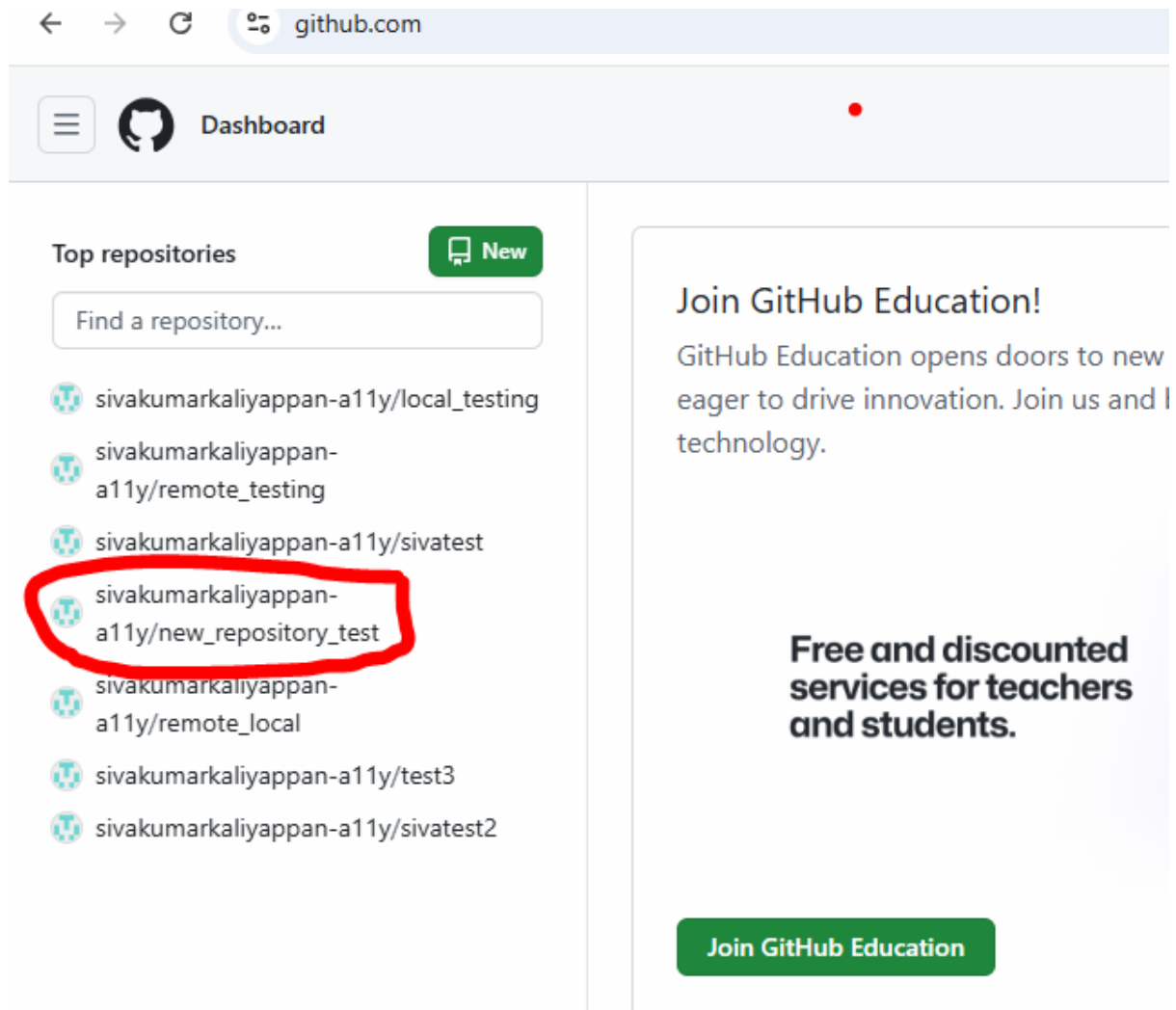
Go to check your desktop in the above path



Alternatively, you can clone an existing repository from a remote source using:

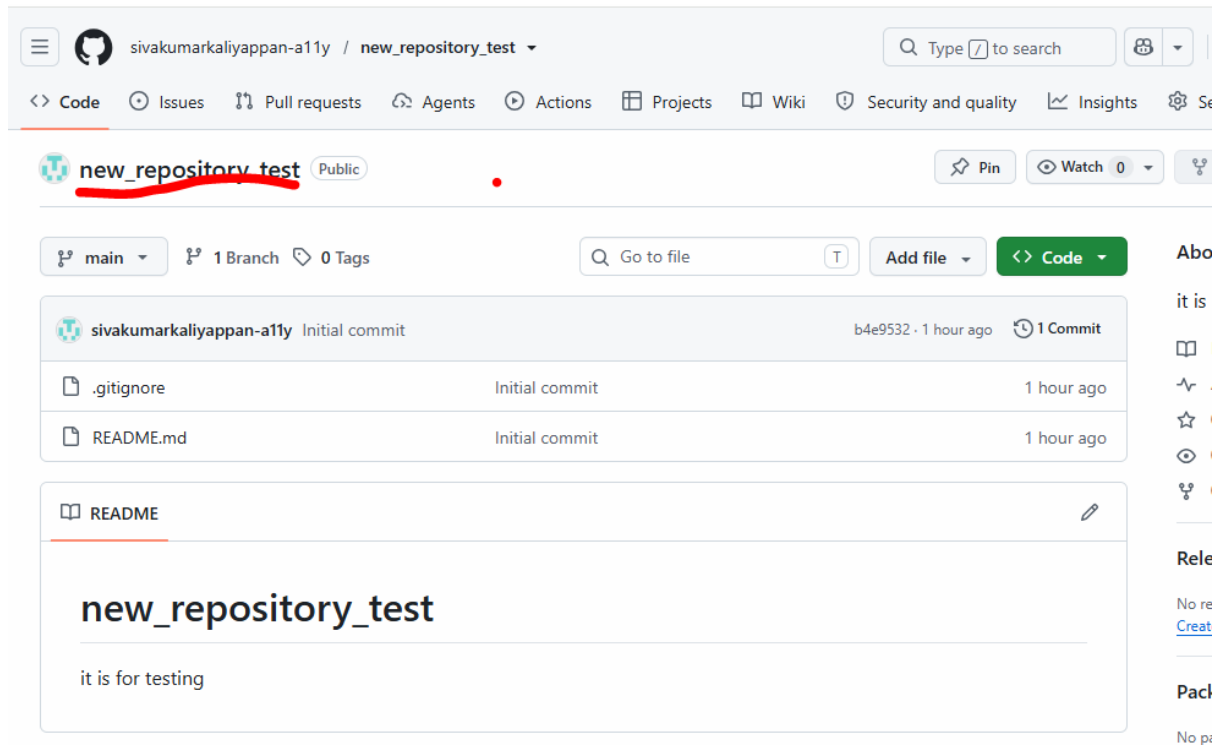
```
git clone https://github.com/username/repository.git
```

to do the above command you need the user name of the repository to find it do the following steps

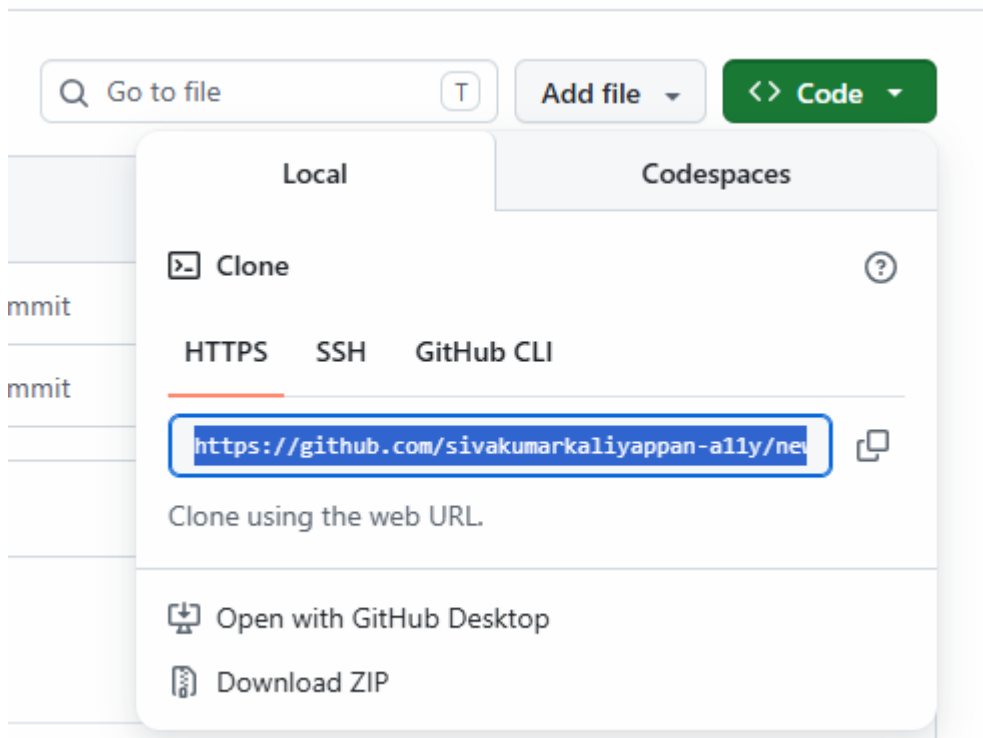


go to above page in which you have already created new repository and click that repository link it will show as follows

MLOPS LAB



Click code tab and find HTTPS and copy the link provided as follows and copy that link



```
git clone https://github.com/username/repository.git
```

replace **user name** into the address you have copied from the HTTPS tab

https://github.com/sivakumarkaliyappan-a11y/new_repository_test.git

```
git clone https://github.com/https://github.com/sivakumarkaliyappan-a11y/new_repository_test.git/
```

run the above command in git bash terminal it will show as follows

MINGW64; c:/Users/DESk

```
DESk@DESKTOP-13UUH3A MINGW64 ~
$ git config --global user.name "sivakuar"

DESk@DESKTOP-13UUH3A MINGW64 ~
$ git config --global user.email "sivakumar.klaiyappan@rajalakshmi.edu.in"

DESk@DESKTOP-13UUH3A MINGW64 ~
$ git init
Initialized empty Git repository in C:/Users/DESk/.git/

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git clone https://github.com/https://github.com/sivakumarkaliyappan-a11y/new_repository_test.git
Cloning into 'new_repository_test'...
remote: Not Found
fatal: repository 'https://github.com/https://github.com/sivakumarkaliyappan-a11y/new_repository_test.git/' not found

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
```

The above one wil display reote repository not found

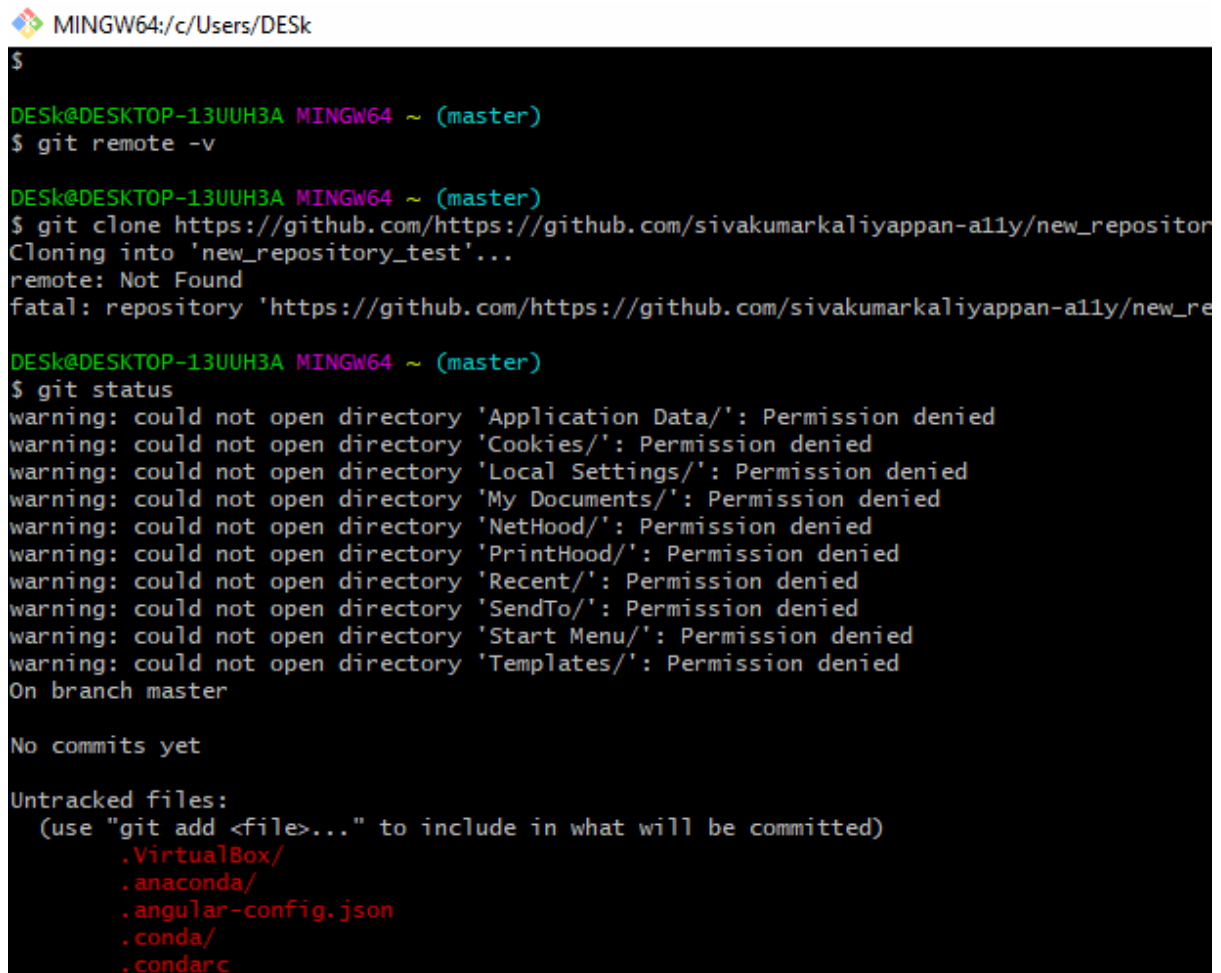
Understand Basic Git Commands

Familiarizing yourself with essential Git commands is crucial for effective collaboration. Here are some key commands:

- **git add:** Stage changes for the next commit.
- **git commit:** Save your staged changes with a descriptive message.
- **git push:** Upload your local commits to the remote repository.
- **git pull:** Fetch and merge changes from the remote repository into your local branch.

- **git branch:** List, create, or delete branches.
- **git checkout:** Switch between branches.
- **git merge:** Combine the changes from different branches.

Git Status command : Displays modified files, staged files, and untracked files in your working environment.



```

MINGW64:/c/Users/DESk
$
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git remote -v

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git clone https://github.com/https://github.com/sivakumarkaliyappan-a1ly/new_repositor
Cloning into 'new_repository_test'...
remote: Not Found
fatal: repository 'https://github.com/https://github.com/sivakumarkaliyappan-a1ly/new_re

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .VirtualBox/
        .anaconda/
        .angular-config.json
        .conda/
        .condarc
  
```

GIT Local Project Setup

Before working with a team, you must configure your local environment and initialize your project.

`git config --global user.name "Your Name"`
Sets your identity.

`git config --global user.email "email@example.com"`
Links commits to you.

git init

Initializes local tracking.

 MINGW64:/c:/Users/DESk

```
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git config --global user.name "Sivakumar"

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git config --global user.email "sivakumar.kaliyappan@rajalakshmi.edu.in"

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git init
Reinitialized existing Git repository in C:/Users/DESk/.git/

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
```

git status

Checks untracked files.

```
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .VirtualBox/
    .anaconda/
    .angular-config.json
    .conda/
    .condarc
    .continuum/
    .copilot/
    .gitconfig
    .idlerc/
    .ipynb_checkpoints/
    .ipython/
    .jupyter/
    .miniconda3/
```

git add <file>

Stages specific changes.

 MINGW64:/c/Users/DESk

```
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git add first.c
fatal: pathspec 'first.c' did not match any files

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
```

It will display If no file found

```
$ git init
```

Git repository created

 MINGW64:/c/Users/DESk

```
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git init
Reinitialized existing Git repository in C:/Users/DESk/.git/

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ touch alpha.html
```

```
$ touch alpha.html
```

```
$ git status
```

Untracked files: `alpha.html`

MLOPS LAB

```
DESK@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .VirtualBox/
    .anaconda/
    .angular-config.json
    .conda/
    .condarc
    .continuum/
    .miniconda3/
```

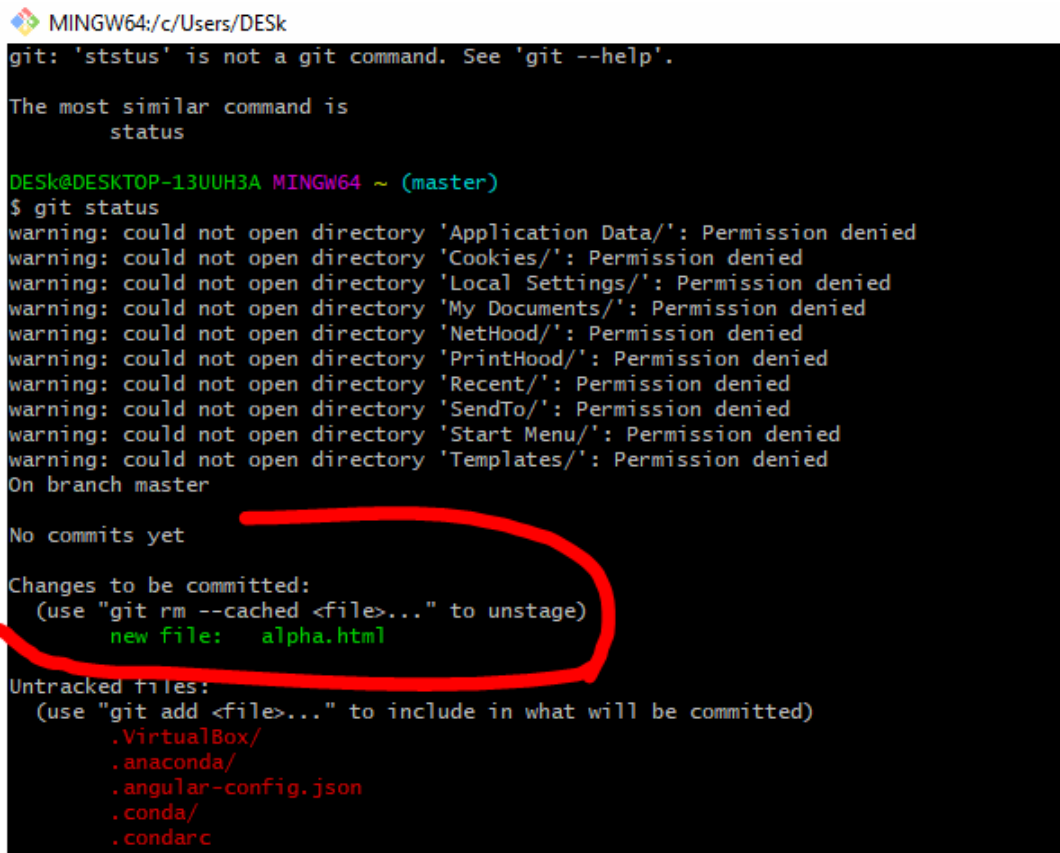
git add alpha.html

```
$
```

```
$ git status
```

```
Changes to be committed:
```

```
new file:   alpha.html
```



```

MINGW64:/c/Users/DESk
git: 'ststatus' is not a git command. See 'git --help'.

The most similar command is
    status

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   alpha.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .VirtualBox/
    .anaconda/
    .angular-config.json
    .conda/
    .condarc
  
```

The **git commit** command captures a local snapshot of your project's currently staged changes and permanently saves it to the Git repository history. Think of it as a checkpoint or a permanent "save point" in a video game

The Two-Step Commit Workflow

In Git, changes do not move directly from your folder into a commit. They follow a two-step process:

1. **Stage your files:** Tell Git which modified files you want to include in the snapshot using `git add`.
2. **Commit your files:** Permanently save those staged changes to your local history using `git commit`.

Common Git Commit Commands

- **git commit -m "your message"**
The most common way to commit. The `-m` flag allows you to write a quick, inline message explaining what changes you made.
- **git commit -am "your message"**
A shortcut flag that automatically stages all *modified* and *deleted* files, and commits them in one step. Note: This **will not** include brand-new, untracked files; you must use `git add` for those first.

- **git commit**

Running the command without any flags opens your system's default text editor (like Vim, Nano, or VS Code) to write a detailed, multi-line commit message.

git commit --amend -m "new message"

Modifies your very last commit. Use this if you made a typo in your last message or forgot to stage a minor change.

git commit --allow-empty -m "message"

Creates a commit with absolutely no file changes. This is highly useful for testing CI/CD pipelines or marking project milestones.

```
$ git commit -m "1st commit"
```

```
[master (root-commit) ee335da] 1st commit
```

 MINGW64:/c/Users/DESk

```
DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$ git commit -m "1st commit"
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Enumerating cruft objects: 1087, done.
Traversing cruft objects: 1087, done.
Counting objects: 100% (1087/1087), done.
Delta compression using up to 4 threads
Compressing objects: 100% (1071/1071), done.
Writing objects: 100% (1087/1087), done.
Total 1087 (delta 227), reused 0 (delta 0), pack-reused 0 (from 0)
[master (root-commit) 3bfe7fa] 1st commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 alpha.html

DESk@DESKTOP-13UUH3A MINGW64 ~ (master)
$
```

`git commit -m "message"`
Saves staged snapshots.

git push: Upload your local commits to the remote repository.

Pushing commits to a remote repository

Use `git push` to push commits made on your local branch to a remote repository.

About `git push`

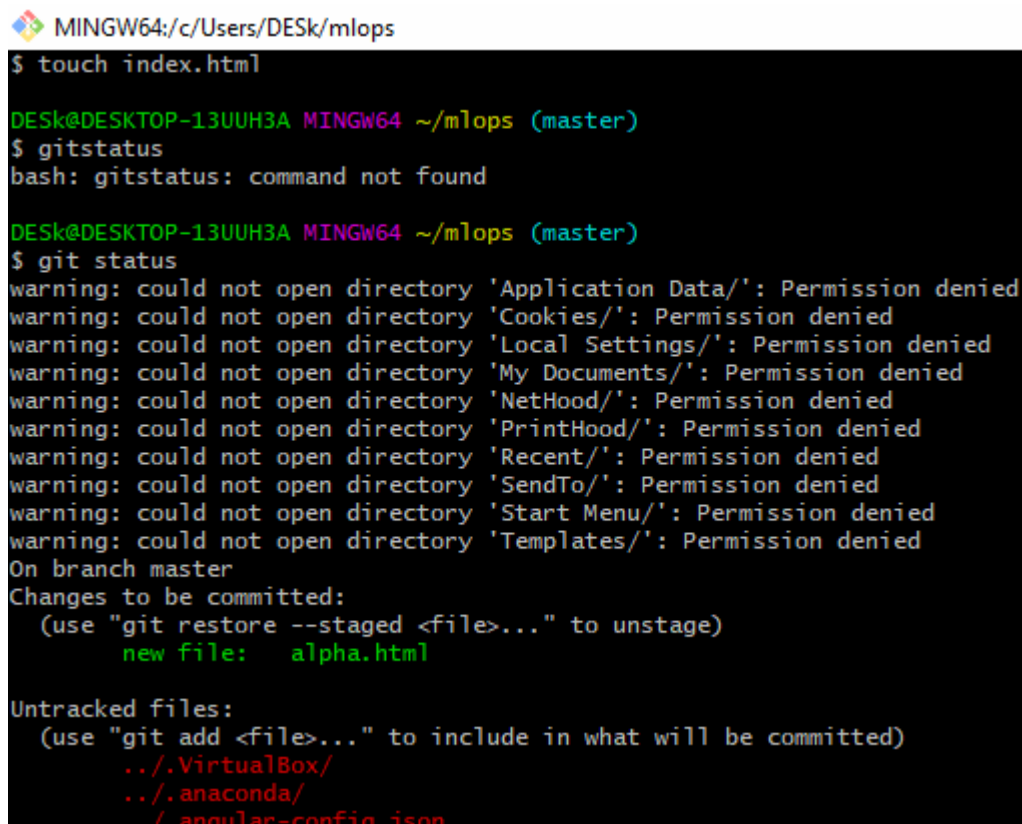
The `git push` command takes two arguments:

- A remote name, for example, `origin`
- A branch name, for example, `main`

For example:

```
git push REMOTE-NAME BRANCH-NAME
```

As an example, you usually run `git push origin main` to push your local changes to your online repository.



```
MINGW64:/c/Users/DESk/mlops
$ touch index.html

DESk@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ gitstatus
bash: gitstatus: command not found

DESk@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   alpha.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  ../.VirtualBox/
  ../.anaconda/
  /angular-config.json
```

MLOPS LAB

MINGW64:/c/Users/DESk/mlops

```
DESk@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git add index.html

DESk@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   alpha.html
    new file:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    ../.VirtualBox/
    ../.anaconda/
    ../.angular-config.json
    ../.conda/
    ../.condarc
    ../.continuum/
    ../.copilot/
```

MINGW64:/c/Users/DESk/mlops

```
DESk@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git remote add origin https://github.com/sivakumarkaliyappan-a1ly/new_repository_test.git
error: remote origin already exists.

DESk@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ https://github.com/sivakumarkaliyappan-a1ly/mlops.git
bash: https://github.com/sivakumarkaliyappan-a1ly/mlops.git: No such file or directory

DESk@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$
```

MLOPS LAB

MINGW64:/c/Users/DESK/mlops

```
DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git remote add origin https://github.com/sivakumarkaliyappan-ally/new_repository_test.git
error: remote origin already exists.

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ https://github.com/sivakumarkaliyappan-ally/mlops.git
bash: https://github.com/sivakumarkaliyappan-ally/mlops.git: No such file or directory

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ https://github.com/sivakumarkaliyappan-ally/
bash: https://github.com/sivakumarkaliyappan-ally/: No such file or directory

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git remote add origin https://github.com/sivakumarkaliyappan-ally/new_repository_test.git/
error: remote origin already exists.

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git remote add origin https://github.com/sivakumarkaliyappan-ally/new_repository_test.git/index.html
error: remote origin already exists.

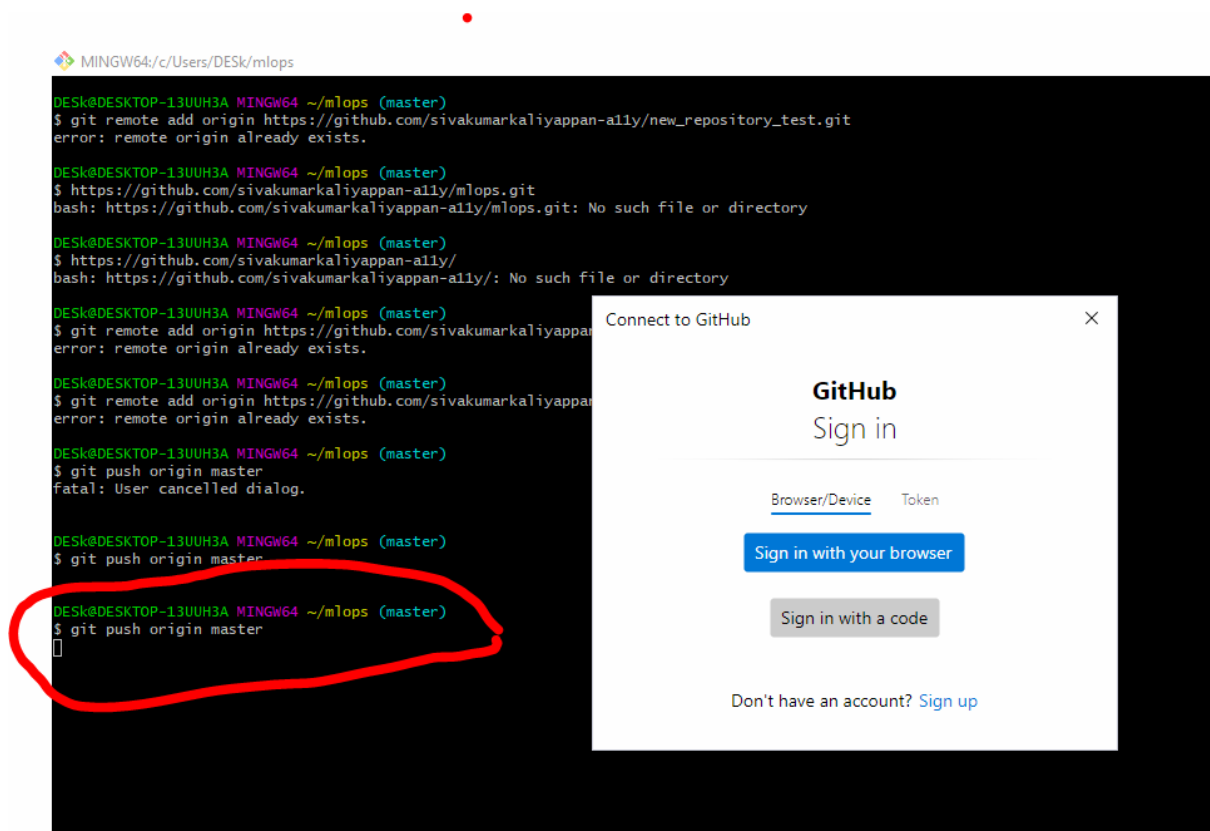
DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git push origin master
fatal: User cancelled dialog.

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$
```

Try the following

git push origin master

if you run the command it will open the following window



Once you have clicked then it will display the following windows

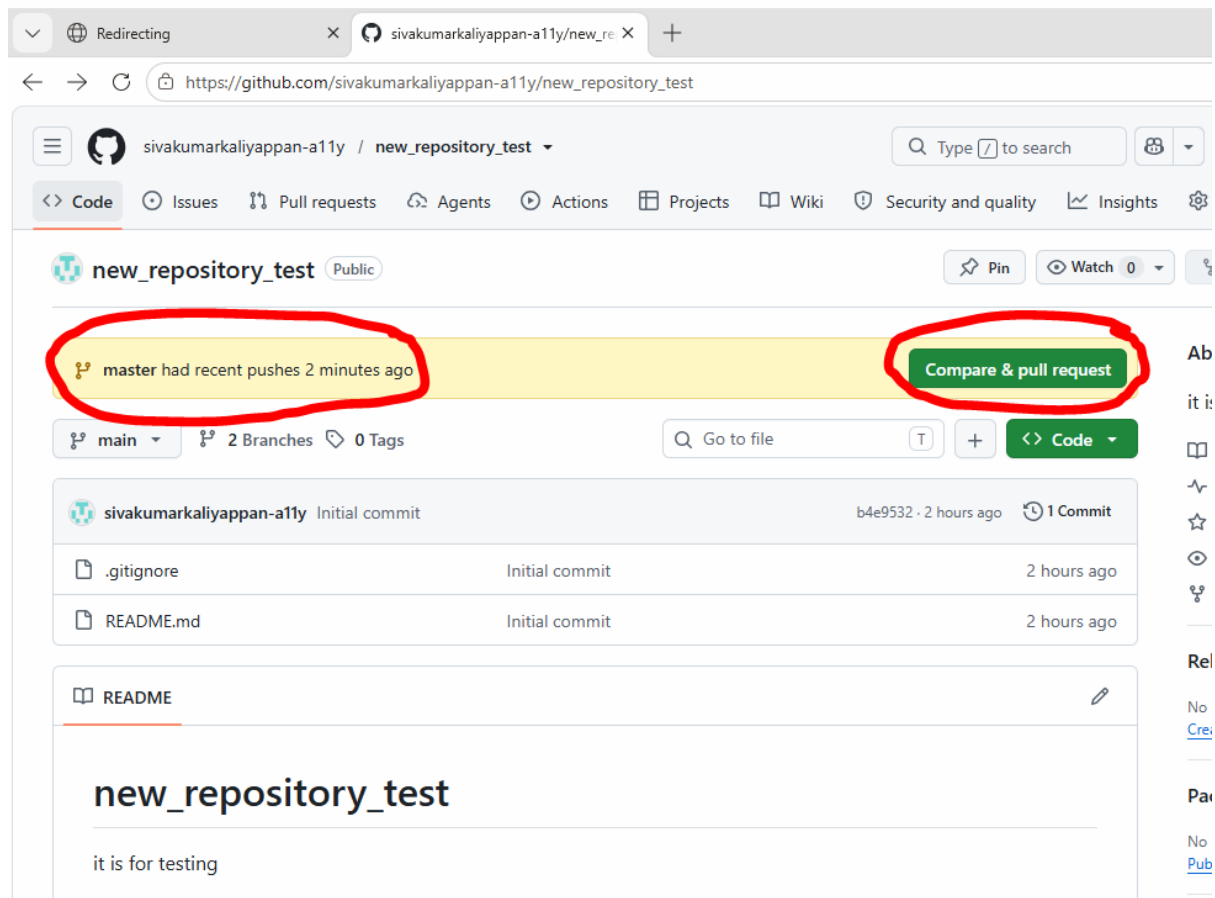
```
DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git push origin master

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git push origin master
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 225 bytes | 225.00 KiB/s, done.
Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/sivakumarkaliyappan-a11y/new_repository_test/pull/new/master
remote:
To https://github.com/sivakumarkaliyappan-a11y/new_repository_test.git
 * [new branch]      master -> master

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git push origin master
Everything up-to-date

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$
```

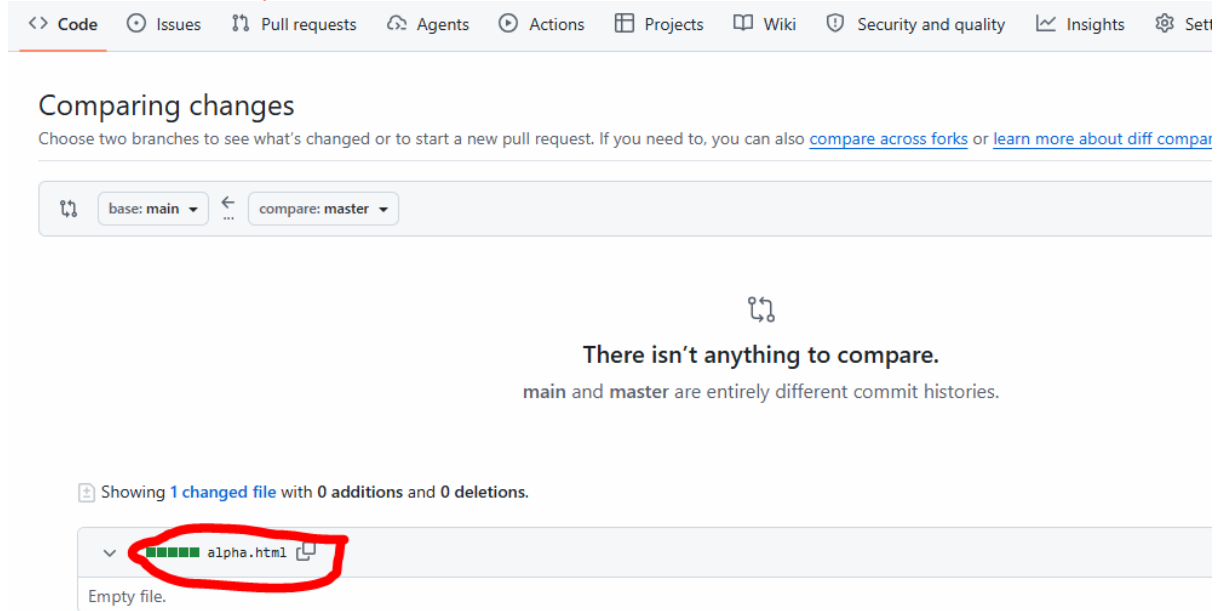
Once done the above command it will appear like this



Once you clicked compare and pull request it will show the **alpha.html**

MLOPS LAB

This is the file where you have created in the local machine



Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff compar](#)

base: main ← compare: master

There isn't anything to compare.
main and master are entirely different commit histories.

Showing 1 changed file with 0 additions and 0 deletions.

alpha.html
Empty file.

```
DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git push origin master
Everything up-to-date

DESK@DESKTOP-13UUH3A MINGW64 ~/mlops (master)
$ git status
warning: could not open directory 'Application Data/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Local Settings/': Permission denied
warning: could not open directory 'My Documents/': Permission denied
warning: could not open directory 'NetHood/': Permission denied
warning: could not open directory 'PrintHood/': Permission denied
warning: could not open directory 'Recent/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
warning: could not open directory 'Start Menu/': Permission denied
warning: could not open directory 'Templates/': Permission denied
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   alpha.html
        new file:   index.html
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../.VirtualBox/
        ../.anaconda/
        ../.angular-config.json
```

